Cryptanalysis of Pseudorandom Error-Correcting Codes

Tianrui Wang[1], Anyu Wang[1](✉), Tianshuo Cong[2](✉), Delong Ran[1], Jinyuan Liu[1], and Xiaoyun Wang[1]

[1] Tsinghua University, Beijing, China
{wangtr22, rdl22, liujinyuan24}@mails.tsinghua.edu.cn,
{anyuwang, xiaoyunwang}@tsinghua.edu.cn
[2] Shandong University, Jinan, Shandong, China
tianshuo.cong@sdu.edu.cn

Abstract. Pseudorandom error-correcting codes (PRC) [7] are a novel cryptographic primitive proposed at CRYPTO 2024. Due to the dual capability of pseudorandomness and error correction, PRC have been recognized as a promising foundational component for watermarking AI-generated content. However, the security of PRC has not been thoroughly analyzed, especially with concrete parameters or even in the face of cryptographic attacks. To fill this gap, we present the first cryptanalysis of PRC. We first propose three attacks to challenge the undetectability and robustness assumptions of PRC. Among them, two attacks aim to distinguish PRC-based codewords from plain vectors, and one attack aims to compromise the decoding process of PRC. Our attacks successfully undermine the claimed security guarantees across all parameter configurations. Notably, our attack can detect the presence of a watermark with overwhelming probability at a cost of 2^{22} operations. We also validate our approach by attacking real-world large generative models such as DeepSeek and Stable Diffusion. To mitigate our attacks, we further propose three defenses to enhance the security of PRC, including parameter suggestions, implementation suggestions, and constructing a revised key generation algorithm. Our proposed revised key generation function effectively prevents the occurrence of weak keys. However, we highlight that the current PRC-based watermarking scheme still cannot achieve a 128-bit security under our parameter suggestions due to the inherent configurations of large generative models, such as the maximum output length of large language models.

Keywords: Pseudorandom error-correcting codes · Large Generative Models · AIGC Watermarking · Meet-in-the-Middle Attack.

1 Introduction

The rapid growth of generative AI models, such as GPT-4o [32] and Stable Diffusion [36], has made AI-Generated Content (AIGC) ubiquitous [17,35]. However, this widespread adoption is accompanied by significant risks of misuse [2,43,26].

Consequently, the identification and provenance tracking of AIGC have become essential requirements for AI governance, with content watermarking standing out as one of the most promising technical solutions [46].

A compelling foundation for such watermarks is the recently developed cryptographic primitive of Pseudorandom Error-correcting Codes (PRCs) [7]. A PRC scheme comprises three algorithms (KeyGen, Encode, Decode). The KeyGen algorithm generates a public/secret key pair. One can encode a message into a codeword using Encode with the public key, while the secret key holder can decode the codeword by Decode to recover the message to perform verification.

PRCs provide two key security properties: robustness, which ensures that decoding a valid codeword using the secret key yields the original message, and undetectability, which requires that codewords are indistinguishable from plain vectors without the secret key.[3] These properties make PRCs suitable for constructing stealthy and robust watermarks for both large language models (LLMs) and generative image models (GIMs).

An instantiation of PRC based on Low-Density Parity-Check (LDPC) codes, referred to as LDPC-PRC, was proposed in [7], offering provable security under standard assumptions, namely the hardness of the Learning Parity with Noise (LPN) and planted XOR problems. This theoretical foundation gives PRC-based watermarking a significant advantage over many heuristic alternatives [45,23,8,11,1,20].

Despite these strong theoretical guarantees, the concrete security of PRCs under practical parameter settings remains unexplored. Since real-world deployment always requires concrete parameter choices, a rigorous security evaluation is urgently needed.

1.1 Our Contributions

This work presents the first concrete security analysis of PRC, bridging the gap between asymptotic security guarantees and practical deployment requirements. Since LDPC-PRC is the only instantiation of PRC to date, our analysis focuses on designing attacks to challenge the undetectability and robustness of LDPC-PRC, consequently facilitating the process of detecting and removing the watermarks of LDPC-PRC-based watermarking schemes.

We propose three attacks. The first one is a key recovery attack that compromises the secret key, thereby enabling an adversary to detect watermarks. The second attack identifies weak keys across multiple targets, allowing watermark detection by exploiting structural biases in the key generation process. In contrast, the third attack directly breaks the robustness property via a noise overlay technique, leading to practical watermark removal. The core procedures of each attack are outlined below.

[3] PRCs also have a soundness property. As discussed in [7], soundness is a technical condition that primarily ensures non-triviality. Our attacks in this paper do not target this property.

Attack-I: Partial Secret Key Recovery. Given a public key, Attack-I aims to recover a portion of its corresponding secret key by the Meet-in-the-Middle technique. Since our goal is to recover only a subset of the public key rows rather than all of them, the constructed sets for collision searching in the Meet-in-the-Middle phase can be made significantly smaller, thus further reducing the complexity. We then mimic the Decode() procedure of LDPC-PRC using the recovered partial key. For any single vector, the Attack-I's advantage of distinguishing an LDPC-PRC codeword from a plain vector remains a constant greater than $1/2$. Consequently, Attack-I can detect watermarks with overwhelmingly high probability given access to a sufficient number of vectors.

Attack-II: Weak Key Distinguisher. This attack exploits an implementation vulnerability in the PRC key generation procedure. We identify that in instantiated applications of LDPC-PRC, such as the LLM watermarking scheme [7] and the GIM watermarking scheme [15], there exists a non-negligible probability of generating public keys that exhibit statistical non-uniformity. Specifically, the adversary is given multiple targets and selects one public key containing duplicated rows as the attack target. The adversary then records the indices of the duplicated rows and checks whether the corresponding positions are identical in the vectors to be tested. If the number of duplicated positions exceeds a predefined threshold, the watermark is detected with overwhelmingly high probability.

Attack-III: Noise Overlay Attack. This attack is based on the observation that the Decode algorithm of PRC is designed to correct a bounded amount of noise to ensure robustness. If an adversary can introduce malicious noise beyond this decoding threshold, watermark removal becomes feasible. However, the key challenge lies in validating the injected noise, as excessive distortion may violate practical constraints in applications such as LLM or GIM (e.g., by degrading output quality). Specifically, since the original noise in PRC is modeled as a Bernoulli-distributed vector, we propose a noise overlay attack that first recovers the original noise vector $\mathbf{e}$, then constructs an adversarial noise vector $\mathbf{e}'$ based on $\mathbf{e}$. The combined noise $\mathbf{e}+\mathbf{e}'$ disrupts the decoding process while adhering to the same error weight, thereby breaking robustness under practical conditions.

We propose a comprehensive analysis of the complexity of each attack, which is summarized in Table 1.

Complexity for Concrete Parameter Configurations. By incorporating specific parameter settings for LDPC-PRC-based LLM watermarking scheme (denoted as Π_{LLM}) and GIM watermarking scheme (denoted as Π_{GIM}) [15], we analyze the concrete complexities of our attacks. For Attack-I, our analysis shows that the complexities are sub-security-level for all parameter choices, where the actual security of Π_{GIM} remains below 56 bits. Attack-II further exposes vulnerabilities resulting from a non-negligible frequency of weak keys across all parameter choices. For example, weak keys appear with a nearly 100% probability when $t = 3, 4$ for Π_{LLM}. For Attack-III, we prove that the security level of Π_{LLM} *cannot exceed* 75 *bits* for any parameter choice with $n \leq 2^{17}$ under the

Table 1: The summary of the attacks. n is the code length of LDPC-PRC, g is the column dimension of the public key $\mathbf{G}$, t is the row weight of the secret key $\mathbf{P}$. The α in Attack-I is calculated in Theorem 1. For Attack-II, P is the probability that a weak key exists. The detailed derivation is provided in Theorem 2. The ε in Attack-III corresponds to the error-correcting capability in Theorem 3.

	Goal	Time Complexity	Additional Requirement
Attack-I	Against Undetectability (Watermark Detection)	$g \cdot \alpha \cdot \binom{n/2}{\lceil t/2 \rceil}$ Theorem 1	-
Attack-II	Against Undetectability (Watermark Detection)	$\mathrm{P}^{-1} \cdot n \cdot g$ Theorem 2	Multi-Target
Attack-III	Against Robustness (Watermark Removal)	$(1/2+\varepsilon)^{-g} \cdot n^3$ Theorem 3	Noise Manipulation

threat of noise overlay attack. Meanwhile, the security level of Π_{GIM} *is below* 60 *bits* with a much lower concrete noise rate.

Evaluation against Real-World Applications. To validate the practicality of our attacks, we choose DeepSeek [16]) and Stable Diffusion [36] as real-world LLM and GIM to implement Π_{LLM} and Π_{GIM}, respectively. To the best of our knowledge, this is the first real-world implementation for Π_{LLM}. However, we validate that Π_{LLM} is impractical for embedding watermarks into LLMs as the entropy of output tokens is too low under reasonable text qualities, rendering the watermark undetectable by the original decoder. Furthermore, evaluation results demonstrate that both the proposed Attack-I and Attack-II can achieve a 100% attack success rate against Π_{LLM} and Π_{GIM} under certain parameter choices (refer to Table 5 and Table 6).

Mitigations. To mitigate the threats posed by the proposed attacks, we propose three defenses. For Attack-I, we suggest the choice of more secure parameters. For Attack-II, we construct a revised key generation algorithm to significantly reduce the probability of weak key occurrence while maintaining the randomness of PRC's keys. For Attack-III, we point out that $n > 2^{24}$ can achieve a 128-bit security. However, the maximum token length of the state-of-the-art LLM is only 2^{15} (i.e., $n > 2^{20}$), which means our attack continues to pose a practical threat on Π_{LLM}. Furthermore, we provide an implementation suggestion for Π_{GIM}, necessitating the inclusion of a noise vector with a larger weight.

1.2 Related Works

Theoretical Studies on PRC. Due to the promising potential of PRC, besides studying how to apply PRC to AIGC watermarking [7,15], several fundamental theoretical studies on PRC have recently emerged, such as proving CCA-security

based on the hardness of LPN [3], discussing unconditionally indistinguishability against space-bounded adversaries [14], and performing a black-box reduction from one-way functions with sub-constant error rates [13]. Unfortunately, these studies have not thoroughly explored the security of PRC. Note that although [7,3] provide a theoretical evaluation of PRC's robustness, it does not conduct a security assessment in conjunction with the concrete parameters.

Watermarking Schemes for AIGC. Watermarks for AIGC can typically be classified into two categories: in-processing schemes and post-processing schemes. Post-processing schemes embed imperceptible yet detectable signals after text generation [38,24] or image generation [9,47,41], inevitably resulting in content quality degradation. Instead, both Π_{LLM} and Π_{GIM} belong to the in-processing approach [21,45,42,44,12], wherein watermark insertion is merged with the process of generating content. With a pseudorandom codeword embedded, the distribution of the watermarked output texts of Π_{LLM} is proven to be identical to that of the original texts, while Π_{GIM} utilizes a watermarked latent that maintains a Gaussian distribution to ensure watermark invisibility in generated images.

Attacks against AIGC Watermarks. To remove watermarks, mainstream methods typically apply heuristic perturbations to the AIGC, such as paraphrasing [22,33] or inserting perturbations [21,19] for the generated text. For images, attackers can adopt image processing operations [40,4] such as photometric distortions or degradation distortions. Unlike these empirical removal attacks, our cryptographic attacks provide provable success rates and well-defined complexity. Note that for watermark detection, Gunn et al. [15] trained a ResNet-18 [18] classifier on watermarked and non-watermarked images, and claim that such a classifier cannot distinguish if generated images contain watermarks of Π_{GIM}. However, we propose two different attacks to challenge the undetectability of Π_{GIM}.

1.3 Organization

In Section 2, we introduce the basic preliminaries and notations of PRC, Π_{LLM}, and Π_{GIM}. Section 3 details the specific procedures of our three attacks. Section 4 analyzes the complexities of our attacks under concrete parameters. The practical experiments on real-world LLMs and GIMs are described in Section 5. Section 6 discusses the effectiveness and side effects of several mitigations. Finally, Section 7 summarizes the paper and outlines several open problems.

2 Preliminaries

2.1 Pseudorandom Error-correcting Codes

This subsection presents the formal definition of PRC and its instantiation based on LDPC codes. Note that in this work, we focus on the zero-bit public-key PRC due to its broader range of applications. The term "zero-bit" refers to a scheme that can encode only a single message. In this scheme, the decoder's

purpose is to determine whether a given input is associated with the keys or not. The multi-bit version is built upon this zero-bit variant; thus, its security directly depends on that of the zero-bit case, making the extension of our attacks to the multi-bit scenario straightforward. Meanwhile, Christ and Gunn [7] also construct two types of secret-key PRCs, but these constructions exhibit inherent limitations. Specifically, the maximum error rate of the construction based on pseudorandom function is bounded by $1/l$, which is significantly smaller than that of the public-key construction with a claimed error-correcting capability for any constant smaller than $1/2$, and the Permuted-PRC based on a permuted code cannot be proven pseudorandom.

Definition 1 (PRC). *Let Σ be a fixed alphabet and $\mathcal{E} : \Sigma^* \to \Sigma^*$ be a noisy channel modeled by adding a noise vector with a Bernoulli distribution, and $k(\lambda) \in \mathbb{N}$ be the length of the message under any fixed λ. A PRC is a triple of polynomial-time randomized algorithms* $(\mathsf{KeyGen}, \mathsf{Encode}, \mathsf{Decode})$ *that satisfy*

- *(**Robustness**) For any message $\mathsf{m} \in \Sigma^{k(\lambda)}$ and any $\lambda \in \mathbb{N}$,*

$$\Pr_{(\mathsf{sk},\mathsf{pk})\leftarrow\mathsf{KeyGen}(1^\lambda)}[\mathsf{Decode}(1^\lambda, \mathsf{sk}, \mathcal{E}(\mathbf{x} \leftarrow \mathsf{Encode}(1^\lambda, \mathsf{pk}, \mathsf{m}))) = \mathsf{m}] \geq 1-\mathsf{negl}(\lambda).$$

- *(**Soundness**) For any fixed $\mathbf{c} \in \Sigma^*$,*

$$\Pr_{(\mathsf{sk},\mathsf{pk})\leftarrow\mathsf{KeyGen}(1^\lambda)}[\mathsf{Decode}(1^\lambda, \mathsf{sk}, \mathbf{c}) = \perp] \geq 1 - \mathsf{negl}(\lambda).$$

- *(**Undetectability**) For any polynomial-time adversary $\mathcal{A}$, let $\Pr[\mathsf{Enc}] = \Pr[\mathcal{A}^{\mathsf{Encode}(1^\lambda,\mathsf{pk},\cdot)}(1^\lambda, \mathsf{pk}) = 1]$ and $\Pr[\mathsf{Uni}] = \Pr[\mathcal{A}^{\mathcal{U}}(1^\lambda, \mathsf{pk}) = 1]$, then*

$$|\Pr_{(\mathsf{sk},\mathsf{pk})\leftarrow\mathsf{KeyGen}(1^\lambda)}[\mathsf{Enc}] - \Pr_{\substack{(\mathsf{sk},\mathsf{pk})\leftarrow\mathsf{KeyGen}(1^\lambda)\\ \mathcal{U}}}[\mathsf{Uni}]| \leq \mathsf{negl}(\lambda).$$

  where $\mathcal{A}^{\mathcal{U}}$ denotes an adversary with access to a uniform random oracle along with previous queries.

Construction 1 ($\mathsf{LDPC\text{-}PRC}$)**.** *Based on LDPC codes, Christ and Gunn [7] construct an instantiation of PRC named* $\mathsf{LDPC\text{-}PRC}$ *as follows.*

- $\mathsf{KeyGen}(1^\lambda)$*:*
  1. *Sample a random matrix $\mathbf{P} \in \mathbb{F}_2^{r\times n}$ where each row of $\mathbf{P}$ has weight t.*
  2. *Sample a random matrix $\mathbf{G} \in \mathbb{F}_2^{n\times g}$ where $\mathbf{PG} = \mathbf{0}$.*
  3. *Sample a random vector $\mathbf{z} \in \mathbb{F}_2^n$.*
  4. *Output public key $\mathsf{pk} = (\mathbf{G}, \mathbf{z})$ and secret key $\mathsf{sk} = (\mathbf{P}, \mathbf{z})$.*
- $\mathsf{Encode}(1^\lambda, (\mathbf{G}, \mathbf{z}))$*:*
  1. *Sample a random vector $\mathbf{s} \in \mathbb{F}_2^g$ and $\mathbf{e} \leftarrow \mathrm{Ber}(n, \omega)$.*
  2. *Output the codeword $\mathbf{x} \leftarrow \mathbf{Gs} + \mathbf{e} + \mathbf{z}$.*
- $\mathsf{Decode}(1^\lambda, (\mathbf{P}, \mathbf{z}), \mathbf{x})$*: If $w_H(\mathbf{P}(\mathbf{x}+\mathbf{z})) \leq (1/2-r^{-1/4})\cdot r$, output 1; otherwise output $\perp$.*

Algorithm 1 `LLM Generation`

Input: $\mathsf{prompt} \in \mathcal{T}^*$
Parameter: Temperature $\mathsf{temp} \in \mathbb{R}^+$, generation length $\mathsf{len} \in \mathbb{Z}^+$
Output: Generated text $T \in \mathcal{T}^*$

$T \leftarrow \perp$
for $i \leftarrow 0$ to $\mathsf{len} - 1$ **do**
  $\mathbf{l} \leftarrow \mathsf{LLM}(\mathsf{prompt} \parallel T)$ ▷ The logits of tokens
  $\mathbf{p} \leftarrow \mathsf{Softmax}(\mathbf{l}/\mathsf{temp})$ ▷ The probabilities of tokens
  $t \xleftarrow{\mathsf{P}} \mathcal{T}$ ▷ Sample the next token
  $T \leftarrow T \parallel t$
end for
return T

Algorithm 2 `GIM Generation`

Input: $\mathsf{prompt} \in \mathcal{T}^*$
Parameter: De-noising model `U-net`, Autoencoder model (`E-net`, `D-net`), de-noising steps s, a (possibly randomized) de-noising scheduler $f(\cdot)$
Output: Generated image Img

$\mathbf{y}^s \xleftarrow{\$} \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$
for $i \leftarrow s$ down to 1 **do**
  $\mathbf{y}^{i-1} \leftarrow f(\texttt{U-net}(\mathsf{prompt}, \mathbf{y}^i, i), i)$
end for
$\mathsf{Img} \leftarrow \texttt{D-net}(\mathbf{y}^0)$
return Img

2.2 Generative Models

The formal definitions and constructions of large language models (LLMs) and generative image models (GIMs) are as follows.

Definition 2 (LLM)**.** *A large language model* LLM *over a token set* $\mathcal{T}$ *is a probabilistic algorithm that takes a prompt* $\mathsf{prompt} \in \mathcal{T}^*$ *as input and generates succeeding text* $T \in \mathcal{T}^{\mathsf{len}}$*. The generation process of* LLM *is detailed in Algorithm 1.*

Definition 3 (GIM)**.** *A generative image model* GIM *is a probabilistic algorithm that takes a prompt* $\mathsf{prompt} \in \mathcal{T}^*$ *as input and outputs an image* $\mathsf{Img} \in [0, 255]^{c \times w \times h}$*. The generation process of* GIM *is detailed in Algorithm 2.*

2.3 PRC-Based Watermarking Schemes

In this subsection, we formalize the key properties of a watermarking scheme for generative models based on PRC in Definition 4. Informally, *robustness* captures the difficulty of removing the watermark under bounded disturbance, *soundness* requires a negligible false positive rate, and *undetectability* means that no efficient adversary can detect the presence of a watermark with non-negligible advantage.

Definition 4 (Watermarking Scheme). *Let* Σ *be a fixed alphabet and* $\mathcal{E} : \Sigma^* \to \Sigma^*$ *be a channel with the security parameter* λ*, a watermarking scheme is a tuple of polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Wat}, \mathsf{Detect})$ *that satisfy*

– *(Robustness) For any content* $x \leftarrow \mathsf{Wat}(\mathsf{prompt})$*,*

$$\Pr_{(\mathsf{sk},\mathsf{pk}) \leftarrow \mathsf{Setup}(1^\lambda)} [\mathsf{Detect}_{\mathsf{sk}}(\mathcal{E}(x)) = \mathsf{true}] \geq 1 - \mathsf{negl}(\lambda).$$

Algorithm 3 `Token Sampling`

Input: Token probabilities $\mathbf{p}$
codeword trunk $\mathbf{x}'$
Parameter: token space $\mathcal{T}$
Output: Sampled token $t \in \mathcal{T}$

$b \leftarrow \perp$
for $i \leftarrow 0$ to $\lceil \log_2 |\mathcal{T}| \rceil - 1$ **do**
  $p' \leftarrow \frac{\sum_{t \in \mathcal{T} \wedge \mathsf{Unpack}(t) = b \| 1 \|_*} p_t}{\sum_{t \in \mathcal{T} \wedge \mathsf{Unpack}(t) = b \|_*} p_t}$
  ▷ Marginal probability of current bit
  if $p' \leq 1/2$ **then**
    $t' \leftarrow \mathrm{Ber}(2p'x'_i)$
  else
    $t' \leftarrow \mathrm{Ber}(1 - 2(1-p')(1-x'_i))$
  end if
  $b \leftarrow b \,\|\, t'$ ▷ Sample token bit-wise
end for
$t \leftarrow \mathsf{Packbits}(b)$
return t

Algorithm 4 `TextToCodeword`

Input: Text $T \in \mathcal{T}^{\mathsf{len}}$
Output: Recovered codeword $\mathbf{x}' \in \mathbb{R}^n$

$\mathbf{x}' = \perp$
for $i \leftarrow 0$ to $\mathsf{len} - 1$ **do**
  $\mathbf{x}' \leftarrow \mathbf{x}' \,\|\, \mathsf{Unpackbits}(T_i)$
end for
return $\mathbf{x}'$

Algorithm 5 `ImageToCodeword`

Input: Image Img
Parameter: De-noising model `U-net`, Autoencoder model (`E-net`, `D-net`), inversion steps s, a (possibly randomized) noise inversion scheduler $z(\cdot)$, error calibration factor σ
Output: Recovered codeword $\mathbf{x}' \in \mathbb{R}^n$

$\mathbf{y}^0 \leftarrow \texttt{E-net}(\mathsf{Img})$
for $i \leftarrow 0$ to $s - 1$ **do**
  $\mathbf{y}^{i+1} \leftarrow z(\texttt{U-net}(\mathbf{y}^i, \perp, i+1), i+1)$
end for
$\mathbf{x}' = \mathsf{erf}(\mathbf{y}^s / \sqrt{2\sigma^2(1+\sigma^2)})$
return $\mathbf{x}'$

– *(Soundness) For any fixed* $c \in \Sigma^*$,

$$\Pr_{(\mathsf{sk},\mathsf{pk}) \leftarrow \mathsf{Setup}(1^\lambda)}[\mathsf{Detect}_{\mathsf{sk}}(c) = \mathsf{false}] \geq 1 - \mathsf{negl}(\lambda).$$

– *(Undetectability) For any polynomial-time adversary* $\mathcal{A}$ *mimics* Detect*, let* Uni *be an oracle that outputs random vector in* Σ^**, and* $x \xleftarrow{\$} \mathsf{Uni}()$ *with probability* $1/2$ *while* $x \xleftarrow{\$} \mathsf{Wat}_{\mathsf{pk}}(\mathsf{prompt})$ *with probability* $1/2$*, then*

$$\left| \Pr_{(\mathsf{sk},\mathsf{pk}) \leftarrow \mathsf{Setup}(1^\lambda)}[\mathsf{Detect}_{\mathsf{sk}}(x) = \mathcal{A}_{\mathsf{pk}}(x)] - \frac{1}{2} \right| \leq \mathsf{negl}(\lambda),$$

Next, we present the constructions of the LDPC-PRC-based LLM watermarking scheme (denoted as Π_{LLM}) and GIM watermarking scheme (denoted as Π_{GIM}).

Construction 2 (Π_{LLM} [7])**.** *A watermarking scheme* Π_{LLM} *for* LLM *over* $\mathcal{T}$ *is a tuple of polynomial-time algorithms* (Setup, Wat, Detect).

– $\mathsf{Setup}(1^\lambda)$ *outputs* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{LDPC\text{-}PRC.KeyGen}(1^\lambda)$*. The matrices are sampled through Algorithm 6.*

Algorithm 6 Key Generation for Π_{LLM} in [7]

Input: Parameters n, $r = 0.99n$, g, and t
Output: Matrices $\mathbf{P}$ and $\mathbf{G}$ for watermark generation

1: Sample a uniformly random matrix $\mathbf{G_0} \leftarrow \mathbb{F}_2^{0.01n \times g}$
2: Initialize an empty list for rows of $\mathbf{P}$
3: **for all** $i \in [0.99\,n]$ **do**
4: Sample a random $(t-1)$-sparse vector $\mathbf{s}_i \in \mathbb{F}_2^{0.01n}$
5: Compute $\mathbf{G}_i$ by appending the row $\mathbf{s}_i^T \mathbf{G}_0$ to $\mathbf{G}_{i-1}$:

$$\mathbf{G}_i = \begin{bmatrix} \mathbf{G}_{i-1} \\ \mathbf{s}_i^T \mathbf{G}_0 \end{bmatrix}$$

6: Construct $\mathbf{s}'_i = [\mathbf{s}_i^T, 0^{i-1}, 1, 0^{0.99n-i}]$
7: Append $\mathbf{s}'_i$ to the list of rows for $\mathbf{P}$
8: **end for**
9: Let $\mathbf{P}$ be the matrix whose rows are $\mathbf{s}'_1, \ldots, \mathbf{s}'_{0.99n}$
10: Let $\mathbf{G} = \mathbf{G}_{0.99n}$
11: Sample a random permutation $\mathbf{\Pi} \in \mathbb{F}_2^{n \times n}$ and let $\mathbf{P} \leftarrow \mathbf{P\Pi^{-1}}, \mathbf{G} \leftarrow \mathbf{\Pi G}$
12: **return** $(\mathbf{P}, \mathbf{G})$

- $\mathsf{Wat}_{\mathsf{pk}}(\mathsf{prompt})$ *is a randomized algorithm that generates watermarked responses. The entire response generation procedure is similar to Algorithm 1, except for the token sampling process (Line 5). Specifically,* Π_{LLM} *first outputs a codeword* $\mathbf{x} \leftarrow \mathsf{LDPC\text{-}PRC.Encode}(1^\lambda, \mathsf{pk})$*, where* $|\mathbf{x}| = \mathsf{len} * \lceil \log_2 |\mathcal{T}| \rceil$*. Then, this codeword is split as* len *trunks and guides the token sampling process in a bit-wise manner through Algorithm 3.*
- $\mathsf{Detect}_{\mathsf{sk}}(T)$ *is an algorithm that outputs* true *or* false*.* Π_{LLM} *first extracts the codeword* $\mathbf{x}'$ *following Algorithm 4, then runs* $\mathsf{LDPC\text{-}PRC.Decode}(1^\lambda, \mathsf{sk}, \mathbf{x}')$*. If the result is* 1*, this algorithm outputs* true*, indicating the watermark is present; otherwise outputs* false*.*

Construction 3 (Π_{GIM} [15])**.** *A watermarking scheme* Π_{GIM} *for a model* GIM *is a tuple of polynomial-time algorithms* $(\mathsf{Setup}, \mathsf{Wat}, \mathsf{Detect}, \mathsf{Decode}')$ *where*

- $\mathsf{Setup}(1^\lambda)$ *outputs* $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{LDPC\text{-}PRC.KeyGen}(1^\lambda)$*. The matrices are sampled in a way similar to Algorithm 6, which will be discussed later.*
- $\mathsf{Wat}_{\mathsf{pk}}(\mathsf{prompt})$ *is a randomized algorithm that outputs a watermarked image* $\widetilde{\mathsf{Img}}$*. To this end,* Π_{GIM} *uses a watermarked latent* $\tilde{\mathbf{y}}$ *as* $\mathbf{y}^s$ *of Algorithm 2 to generate images. The generation process of* $\tilde{\mathbf{y}}$ *is*

$$\tilde{\mathbf{y}} = (\tilde{y}_1, \cdots, \tilde{y}_n) \quad \textit{where} \quad \tilde{y}_i = (1 - 2x_i)\,|y_i|,$$

  where $\mathbf{y} = (y_1, \cdots, y_n) \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_n)$ *and* $\mathbf{x} \leftarrow \mathsf{LDPC\text{-}PRC.Encode}(1^\lambda, \mathsf{pk})$*.*
- $\mathsf{Detect}_{\mathsf{sk}}(\mathsf{Img})$ *is an algorithm that outputs whether* Img *is watermarked.* Π_{GIM} *first extract a codeword* $\mathbf{x}'$ *from* Img *through Algorithm 5, and then run* $\mathsf{LDPC\text{-}PRC.Decode}(1^\lambda, \mathsf{sk}, \mathbf{x}')$*. If the result is* 1*,* Detect *outputs* true*, and* false *vise versa.*

Remark. We highlight that there are two main differences between Π_{GIM} and Π_{LLM}. First, Π_{GIM} initializes the key pair by generating a sparse vector $\mathbf{s}_i \in \mathbb{F}_2^{(n-r+i)}$ instead of $\mathbb{F}_2^{n-r}$ (Line 4 of Algorithm 6). Second, in order to embed extra messages and estimate the false positive rate (FPR), Π_{GIM} increases the column number of $\mathbf{G}$ from g to k by adding message bits and parity-check bits, which could be seen as a multi-bit PRC with its own implementation. Besides detecting the watermark, Π_{GIM} also introduces a $\mathsf{Decode}'()$ function to recover the original message $\mathbf{s}$ with the belief propagation algorithm, leveraging the sparsity of the secret key $\mathbf{P}$. However, we will demonstrate that such a modification can cause vulnerability, which is detailed in Section 6.2.

3 Attacks Against PRC

This section details the three attacks and provides their complexity analyses.

3.1 Attack-I: Partial Secret Key Recovery

Given a public key $\mathbf{G} \in \mathbb{F}_2^{n\times g}$, Attack-I aims to recover a set of row vectors $\{\mathbf{v}^{(j)}\} \subseteq \mathbb{F}_2^n$ of the secret key $\mathbf{P}$. Using these vectors, an adversary can distinguish whether a set of target vectors $\{\mathbf{x}\}$ is PRC-encoded by computing the distribution of $\langle \mathbf{v}^{(j)}, \mathbf{x}+\mathbf{z}\rangle$. The attack procedure is detailed below and summarized in Algorithm 7.

Constructing the Set of Vectors $\{\mathbf{v}^{(j)}\}$. Since Attack-I aims to compute a set of row vectors $\{\mathbf{v}^{(j)}\} \subseteq \mathbb{F}_2^n$ of the secret key $\mathbf{P}$, which satisfy $\mathbf{v}^{(j)}\mathbf{G} = \mathbf{0}$ and $w_H(\mathbf{v}^{(j)}) = t$. We adapt the information set decoding (ISD) algorithm [5] to recover such a set of short vectors using a Meet-In-The-Middle (MITM) approach.

Let $n_1 = \lceil n/2 \rceil$, $n_2 = n - n_1$, $t_1 = \lceil t/2 \rceil$, and $t_2 = t - t_1$. For simplicity, we assume n is even, so $n_1 = n_2 = n/2$. Let $\mathscr{L}$ be the set of vectors in $\mathbb{F}_2^n$ satisfying $w_H(\mathbf{v}_{1:n_1}) = t_1$ and $w_H(\mathbf{v}_{n_1+1:n}) = t_2$ for all $\mathbf{v} \in \mathscr{L}$. The probability that a random vector of weight t falls into $\mathscr{L}$ is $q = \binom{n_1}{t_1}\binom{n_2}{t_2}/\binom{n}{t}$. Thus, the expected number of rows of $\mathbf{P}$ in $\mathscr{L}$ is $r' = qr$, since each row of $\mathbf{P}$ is independently sampled at random.

To recover the expected r' rows, we construct two sets $\mathscr{L}_1$ and $\mathscr{L}_2$, where $\mathscr{L}_1$ contains all $\binom{n}{t_1}$ possible sums of t_1 distinct rows from the upper half of $\mathbf{G}$, and $\mathscr{L}_2$ contains all $\binom{n}{t_2}$ possible sums of t_2 distinct rows from the bottom half of $\mathbf{G}$:

$$\mathscr{L}_1 = \{(a_1, ..., a_{t_1}, \mathbf{sum}_1) \mid 1 \le a_1 < \cdots < a_{t_1} \le n, \mathbf{sum}_1 = \sum_{i=1}^{t_1} \mathbf{G}_{a_i}\},$$

$$\mathscr{L}_2 = \{(a_{t_1+1}, ..., a_t, \mathbf{sum}_2) \mid 1 \le a_{t_1+1} < \cdots < a_t \le n, \mathbf{sum}_2 = \sum_{i=t_1+1}^{t} \mathbf{G}_{a_i}\},$$

where $\mathbf{G}_{a_i}$ denotes the a_i-th row of $\mathbf{G}$. We then employ the MERGE-JOIN algorithm [5] (a MITM implementation) to construct the target set

$$\mathscr{L} = \{(a_1, ...a_{t_1}, a_{t_1+1}, ..., a_t) \mid \mathbf{sum}_1 + \mathbf{sum}_2 = \mathbf{0}\},$$

Algorithm 7 `Attack-I`

Input: Public Key $(\mathbf{G}, \mathbf{z})$, target codewords $\mathscr{X} = \{\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\}$
Parameter: $(\alpha, \beta, r', l, \tau)$ defined in Section 3.1 and (g, t, n, r) of LDPC-PRC
Output: true or false, where true indicates $\mathscr{X}$ are encoded by PRC and false otherwise

$\mathscr{L}_1 \leftarrow \{(a_1, ..., a_{t_1}, \mathbf{sum}_1) | 1 \le a_1 < ... < a_{t_1} \le n, \mathbf{sum}_1 = \sum_{i=1}^{t_1} \mathbf{G}_{a_i}\}$
$\mathscr{L}_2 \leftarrow \{(a_{t_1+1}, ..., a_t, \mathbf{sum}_2) | 1 \le a_{t_1+1} < ... < a_t \le n, \mathbf{sum}_2 = \sum_{i=t_1+1}^{t} \mathbf{G}_{a_i}\}$
if t is **even then**
    Construct $\mathscr{L}_1' \subset \mathscr{L}_1$ and $|\mathscr{L}_1'| = \frac{1}{\sqrt{r'/l}}|\mathscr{L}_1|$
    Construct $\mathscr{L}_2' \subset \mathscr{L}_2$ and $|\mathscr{L}_2'| = \frac{1}{\sqrt{r'/l}}|\mathscr{L}_2|$
else
    Construct $\mathscr{L}_1' \subset \mathscr{L}_1$ and $|\mathscr{L}_1'| = \alpha|\mathscr{L}_1|$
    Construct $\mathscr{L}_2' \subset \mathscr{L}_2$ and $|\mathscr{L}_2'| = \beta|\mathscr{L}_2|$
end if
Sort $\mathscr{L}_1', \mathscr{L}_2'$
$\mathscr{L}' \leftarrow$ MERGE-JOIN$(\mathscr{L}_1', \mathscr{L}_2')$
Initialize an empty list for vectors of $\mathcal{V}$
for all $(a_1, \cdots, a_t) \in \mathscr{L}'$ **do**
    $\mathbf{v} \leftarrow (v_1, \cdots, v_n)$ ▷ where $v_i = 1$ if $\exists j$ s.t. $i = a_j$ and $v_i = 0$ otherwise
    Append $\mathbf{v}$ to the list of vectors for $\mathcal{V}$
end for
Initialize $N_{\mathsf{tot}}, N_{\mathsf{zero}} \leftarrow 0, 0$
for all $\mathbf{v}^{(j)} \in \mathcal{V}$ and $\mathbf{x}^{(i)} \in \mathscr{X}$ **do**
    $N_{\mathsf{tot}} \leftarrow N_{\mathsf{tot}} + 1$
    if $\langle \mathbf{v}^{(j)}, \mathbf{x}^{(i)} + \mathbf{z} \rangle = 0$ **then**
        $N_{\mathsf{zero}} \leftarrow N_{\mathsf{zero}} + 1$
    end if
end for
if $N_{\mathsf{zero}}/N_{\mathsf{tot}} \ge \tau$ **then**
    return true
else
    return false
end if

where $(a_1, ..., a_{t_1}, \mathbf{sum}_1) \in \mathscr{L}_1$ and $(a_{t_1+1}, ..., a_t, \mathbf{sum}_2) \in \mathscr{L}_2$. Thus, $\mathscr{L}$ stores the r' rows of the secret key $\mathbf{P}$.

To improve efficiency, we reduce $\mathscr{L}_1$ and $\mathscr{L}_2$ to smaller sets, ensuring the expected number of recovered rows from $\mathbf{P}$ is $l < r'$:

– When t is **even**, we randomly choose subsets $\mathscr{L}_i' \subset \mathscr{L}_i$ for $i = 1, 2$, with $|\mathscr{L}_1'| = \lceil \frac{1}{\sqrt{r'/l}} \binom{n_1}{t/2} \rceil$ and $|\mathscr{L}_2'| = \lceil \frac{1}{\sqrt{r'/l}} \binom{n_2}{t/2} \rceil$. Using MITM, we construct:

$$\mathscr{L}' = \{(a_1, ..., a_t) \mid \mathbf{sum_1} + \mathbf{sum_2} = \mathbf{0}\} \subseteq \mathscr{L},$$

where $(a_1, ..., a_{t_1}, \mathbf{sum_1}) \in \mathscr{L}_1'$ and $(a_{t_1+1}, ..., a_t, \mathbf{sum_2}) \in \mathscr{L}_2'$. The expected number of rows of $\mathbf{P}$ in $\mathscr{L}'$ is $\frac{qr}{\sqrt{r'/l}\sqrt{r'/l}} = l$.

– When t is **odd**, we tailor $\mathscr{L}_1'$ and $\mathscr{L}_2'$ according to their sizes. Let $|\mathscr{L}_1'| = \alpha\binom{n/2}{t_1}$ and $|\mathscr{L}_2'| = \beta\binom{n/2}{t_2}$, where $\alpha = \sqrt{\frac{t_1}{(n/2-t_2)r'/l}}$ and $\beta = \frac{(n/2-t_2)\alpha}{t_1}$, ensuring $|\mathscr{L}_1'| = |\mathscr{L}_2'|$ and $\alpha\beta = l/r'$. Similarly, we construct $\mathscr{L}'$ via MITM, with the expected number of rows of $\mathbf{P}$ in $\mathscr{L}'$ equal to $\alpha\beta r' = l$.

Distinguishing Vectors. Suppose the rows $\{\mathbf{v}^{(1)}, \cdots, \mathbf{v}^{(l)}\}$ of the secret key $\mathbf{P}$ are generated in the first step, and the adversary aims to decide whether vectors $\{\mathbf{x}^{(1)}, ..., \mathbf{x}^{(m)}\}$ are LDPC-PRC-encoded or plain vectors. We assume these m vectors are independently sampled. The adversary computes $\{\langle \mathbf{v}^{(j)}, \mathbf{x}^{(i)} + \mathbf{z}\rangle\}_{1\le j\le l, 1\le i\le m}$ and calculates the ratio of zeros in the result set. The adversary then chooses a threshold τ and concludes that the m vectors are LDPC-PRC-encoded if the ratio exceeds τ, and plain vectors otherwise.

If the m vectors are PRC-generated, i.e., $\mathbf{x}^{(i)} + \mathbf{z} = \mathbf{G}\mathbf{s} + \mathbf{e}^{(i)}$ with $\mathbf{e}^{(i)} \leftarrow \mathrm{Ber}(n, \omega)$, then

$$p \coloneqq \Pr[\langle \mathbf{v}^{(j)}, \mathbf{x}^{(i)} + \mathbf{z}\rangle = 0] = \sum_{\tilde{j} \text{ is even}} \binom{t}{\tilde{j}} \cdot \omega^{\tilde{j}} \cdot (1-\omega)^{t-\tilde{j}}.$$

As proved in [7], there exists a constant $\delta > 0$ such that $p > 1/2 + \delta$. The expected true positive rate (TPR), i.e., the probability of correctly identifying m LDPC-PRC-encoded vectors, is:

$$\mathsf{TPR} = \sum_{j \ge \tau \cdot ml} \binom{ml}{j} \cdot p^j \cdot (1-p)^{ml-j},$$

and the false positive rate (FPR), i.e., the probability of incorrectly identifying m plain vectors as encoded, is:

$$\mathsf{FPR} = \sum_{j \ge \tau \cdot ml} \binom{ml}{j} \cdot \left(\frac{1}{2}\right)^{ml}.$$

The adversary can choose m and τ to achieve target TPR and FPR values. For example, to achieve $\mathsf{TPR} = 1 - \mathsf{negl}(\lambda)$ and $\mathsf{FPR} = \mathsf{negl}(\lambda)$, we can set l, m such that $ml = r$ and $\tau = 1/2 + r^{-1/4}$, aligning with the $\mathsf{Decode}()$ function in the original Construction 1.

Theorem 1 (Complexity of Attack-I)**.** *Given a public key* $\mathbf{G}$ *and* $m = \Theta(n)$ *input vectors, there exists an adversary* $\mathcal{A}$ *that can distinguish against the undetectability property of* LDPC-PRC *with advantage* $|\Pr[\mathsf{Enc}] - \Pr[\mathsf{Uni}]| \ge 1 - \mathsf{negl}(\lambda)$ *in time complexity*

$$\mathrm{T}_{\mathsf{partial}} = O(1) \cdot g \cdot \alpha \cdot \binom{n/2}{\lceil t/2 \rceil} \tag{1}$$

and data complexity $\Theta(n)$*, where* $\alpha = 1/\sqrt{qr}$ *for even* t *and* $\alpha = \sqrt{\frac{\lceil t/2 \rceil}{qr(n/2 - \lfloor t/2 \rfloor)}}$ *for odd* t*, with* $q = \binom{n/2}{t_1}\binom{n/2}{t_2}/\binom{n}{t}$.

Proof. The data complexity is clearly $m = \Theta(n)$. For the time complexity, we can set $l = r/m = O(1)$ to achieve advantage $1 - \mathsf{negl}(\lambda)$ as discussed. Attack-I then takes time $\mathrm{T}_{\mathsf{partial}} + \Theta(n)$, where $\mathrm{T}_{\mathsf{partial}}$ is the time to recover rows of $\mathbf{P}$ and $\Theta(n)$ is the distinguishing complexity. Note that $\mathrm{T}_{\mathsf{partial}} = g \cdot \max\{|\mathscr{L}_1'|, |\mathscr{L}_2'|\}$, where the g factor comes from summing rows of $\mathbf{G}$. When t is **even**, $|\mathscr{L}_1'| = |\mathscr{L}_2'| = \frac{1}{\sqrt{r'}}\binom{n/2}{t/2}$, so $\mathrm{T}^{\mathrm{even}}_{\mathsf{partial}} = \frac{g}{\sqrt{r'}} \cdot \binom{n/2}{t/2}$. When t is **odd**, $|\mathscr{L}_1'| = \alpha\binom{n/2}{t_1}$ and $|\mathscr{L}_2'| = \beta\binom{n/2}{t_2}$. To minimize MITM complexity, we set $|\mathscr{L}_1'| = |\mathscr{L}_2'|$, i.e., $\alpha\binom{n/2}{t_1} = \beta\binom{n/2}{t_2}$, with $\alpha\beta = l/r' = O(1)/r'$. Thus, $\mathrm{T}^{\mathrm{odd}}_{\mathsf{partial}} = g \cdot \alpha \cdot \binom{n/2}{t_1}$, where $\alpha = \sqrt{\frac{\lceil t/2 \rceil}{qr(n/2 - \lfloor t/2 \rfloor)}}$. □

Public-Key-Free Variant. Attack-I can also be adapted to the setting in the absence of public keys. In this case, by canceling the one-time pad through pairwise differences of codewords and exploiting the resulting statistical bias, the attacker can still distinguish watermarked codewords from uniformly random ones without access to the public key. More details of this public-key-free variant are provided in Appendix A.1.

3.2 Attack-II: Weak Key Distinguisher

This subsection presents an efficient *multi-target* distinguishing attack that exploits an implementation vulnerability in the $\mathsf{KeyGen}()$ procedure. In a multi-target attack, the adversary attempts to compromise any one vulnerable key from a set of multiple public keys.

Current instantiations of PRC, such as the LLM watermarking scheme [7] and the GIM scheme [15], use Algorithm 6 or its variant to generate the random matrix $\mathbf{P} \xleftarrow{\$} \mathbb{F}_2^{r \times n}$, where each row of $\mathbf{P}$ has weight t. However, we find that rows of $\mathbf{G}$ can exhibit relationships that directly facilitate watermark distinguishing. This attack focuses on the simplest case where some rows of $\mathbf{G}$ are identical, and we call such $\mathbf{G}$ a weak key. The attack procedure is detailed below and summarized in Algorithm 8.

Identifying Weak Keys. Weak keys can be identified by detecting whether $\mathbf{G}$ contains identical rows, which can be accomplished in $O(ng)$ time per key using hash tables. We now analyze the probability that a weak key is generated in the LLM watermarking scheme [7] and the GIM scheme [15].

Let $\mathrm{P}^{\mathrm{LLM}}_{\mathsf{weak}}$ denote the weak key probability for the LLM scheme [7]. Since $\mathbf{G}$ is generated using Algorithm 6, recall that it first samples a uniformly random matrix $\mathbf{G}_0$ with $n - r$ rows, then constructs the remaining r rows iteratively. Index these r rows from 1 to r. Ignoring the permutation $\mathbf{\Pi}$, each of the last r rows of $\mathbf{G}$ is the sum of $t-1$ rows from the first $n-r$ rows of $\mathbf{G}_0$. The number of possible distinct rows is $N = \binom{n-r}{t-1}$. Let p_i denote the probability that the first i rows are distinct. Then $p_1 = 1$ and $p_{i+1} = p_i \cdot \frac{N-i}{N}$. Thus, $p_r = \frac{N(N-1)\cdots(N-r+1)}{N^r}$, and

$$\mathrm{P}^{\mathrm{LLM}}_{\mathsf{weak}} = 1 - \frac{N(N-1)\cdots(N-r+1)}{N^r}. \tag{2}$$

Algorithm 8 `Attack-II`

Input: Public Keys $\{(\mathbf{G}^{(i)}, \mathbf{z}^{(i)})\}$, target codewords $\mathscr{X} = \{\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\}$ outputted by a certain oracle
Parameter: Threshold τ defined in Section 3.1 and (g, t, n, r) of LDPC-PRC
Output: true or false, where true indicates $\mathscr{X}$ are generated by LDPC-PRC and false otherwise

1: Randomly select a public key pair $(\mathbf{G}, \mathbf{z}) \xleftarrow{\$} \{(\mathbf{G}^{(i)}, \mathbf{z}^{(i)})\}$
2: $\mathscr{I} \leftarrow \{(\alpha_j, \beta_j) | \mathbf{G}_{\alpha_j} = \mathbf{G}_{\beta_j}\}$
3: **if** $|\mathscr{I}| = 0$ **then**
4: goto line 1
5: **end if**
6: Obtain $\mathscr{X}$ from the oracle corresponding to $(\mathbf{G}, \mathbf{z})$
7: Initialize $N_{\mathsf{tot}}, N_{\mathsf{dup}} \leftarrow 0, 0$
8: **for all** $(\alpha_j, \beta_j) \in \mathscr{I}$ and $\mathbf{x}^{(i)} \in \mathscr{X}$ **do**
9: $N_{\mathsf{tot}} \leftarrow N_{\mathsf{tot}} + 1$
10: **if** $x^{(i)}_{\alpha_j} = x^{(i)}_{\beta_j}$ **then**
11: $N_{\mathsf{dup}} \leftarrow N_{\mathsf{dup}} + 1$
12: **end if**
13: **end for**
14: **if** $N_{\mathsf{dup}}/N_{\mathsf{tot}} \geq \tau$ **then**
15: **return** true
16: **else**
17: **return** false
18: **end if**

For $\mathrm{P}^{\mathrm{GIM}}_{\mathsf{weak}}$, the key generation implementation differs from Algorithm 6 and has a larger enumeration space. The i-th row is a combination of the previous $i-1$ rows and the $n-r$ rows of $\mathbf{G}_0$. Let p_i denote the probability that the first i rows are distinct. Then $p_1 = 1$ and $p_{i+1} = p_i \cdot \left(1 - \frac{i}{\binom{n-r+i}{t-1}}\right)$. Therefore

$$\mathrm{P}^{\mathrm{GIM}}_{\mathsf{weak}} = 1 - \prod_{i=1}^{r} \left(1 - \frac{i-1}{\binom{n-r+i-1}{t-1}}\right). \tag{3}$$

Distinguishing Vectors. Assume we have l pairs of row indices $\{(\alpha_j, \beta_j)\}$ such that $\mathbf{G}_{\alpha_j} = \mathbf{G}_{\beta_j}$ for all $1 \leq j \leq l$. For any $\mathbf{s} \in \mathbb{F}_2^{g \times 1}$, the α_j-th and β_j-th positions of $\mathbf{Gs}$ are always equal. This holds because $\mathbf{Gs}$ computes a linear combination of the columns of $\mathbf{G}$, and identical rows ensure identical positions in the result.

Although the output $\mathbf{Gs}+\mathbf{e}$ is perturbed by the noise vector $\mathbf{e}$, the adversary can construct a distinguisher with constant advantage. Let $\mathbf{e}_{\alpha_j} \leftarrow \mathrm{Ber}(\rho)$ and $\mathbf{e}_{\beta_j} \leftarrow \mathrm{Ber}(\rho)$ with $\rho < 1/2$. The probability that positions α_j and β_j in $\mathbf{Gs} + \mathbf{e}$ are equal is $\rho^2 + (1-\rho)^2 = \frac{1+(2\rho-1)^2}{2}$. For a random vector, this probability is $\frac{1}{2}$. The statistical difference is $\frac{(2\rho-1)^2}{2}$, which is a constant number.

Using a threshold τ similar to Section 3.1, the adversary can distinguish $\mathbf{Gs} + \mathbf{e}$ from a random vector in linear time, breaking the undetectability of

PRC. In practice, the adversary collects vectors $\{\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(m)}\}$ from the oracle and calculates the ratio of vectors where positions α_j and β_j are equal. If this ratio exceeds τ, the adversary concludes the oracle uses PRC encoding. When $ml = r$ and $\tau = 1/2 + r^{-1/4}$, the distinguishing advantage is overwhelmingly high due to a deduction same to that in Section 3.1.

Theorem 2 (Time Complexity of Attack-II)**.** *There exists an adversary $\mathcal{A}$ such that, for a* fixed *key, runs in time $O(1) \cdot n \cdot g$. If the key is weak, $\mathcal{A}$ correctly identifies it and achieves a distinguishing advantage $|\Pr[\mathsf{Enc}] - \Pr[\mathsf{Uni}]| \geq 1 - \mathsf{negl}(\lambda)$ using $\Theta(n)$ input vectors.*

Since a weak key is encountered after P^{-1} independent keys in expectation, the total expected time complexity is

$$\mathrm{T}_{\mathsf{dis}} = O(1) \cdot \mathrm{P}^{-1} \cdot n \cdot g, \tag{4}$$

where $\mathrm{P} = \mathrm{P}_{\mathsf{weak}}^{\mathrm{LLM}}$ in (2) for the LLM scheme [7], and $\mathrm{P} = \mathrm{P}_{\mathsf{weak}}^{\mathrm{GIM}}$ in (3) for the GIM scheme [15].

Proof. The expected cost of identifying a weak key is $O(1) \cdot \mathrm{P}^{-1} \cdot n \cdot g$, where $O(n \cdot g)$ is the cost of detecting duplicate rows in a single key, and the P^{-1} is the number of expected keys. Given a weak key $\mathbf{G}$, the adversary can distinguish $\mathbf{Gs} + \mathbf{e}$ from a random vector in linear time. A distinguishing advantage of $1 - \mathsf{negl}(\lambda)$ is achieved by processing $\Theta(n)$ input vectors. Therefore, the total expected time complexity is $\mathrm{T}_{\mathsf{dis}} = O(1) \cdot \mathrm{P}^{-1} \cdot n \cdot g + \Theta(n)$. The $\Theta(n)$ term is omitted from the final complexity as it is dominated by the first term. □

Public-Key-Free Variant. Similarly, Attack-II can also be enhanced for the public-key-free scenario. The detailed attacking pipeline is discussed in Appendix A.2.

3.3 Attack-III: Noise Overlay Attack

In PRC, the noise vector is modeled as a Bernoulli noise channel as defined in Definition 1. This noise can originate from multiple sources, such as the inherent noise in LLM or GIM applications. For instance, in LLM watermarking, noise arises during the sampling of t_i from (p_i, x_i) by Algorithm 3, while in GIM, noise is introduced by the noise inversion procedure (i.e., $z(\cdot)$ of Algorithm 5). However, we highlight that an adversary may also actively modify content to evade detection. To investigate this threat, we propose a noise overlay attack that challenges the robustness of PRC. The attack procedure is detailed below and summarized in Algorithm 9.

Algorithm 9 `Attack-III`

Input: Public Key $(\mathbf{G}, \mathbf{z})$, target codeword $\mathbf{x} = \mathbf{Gs} + \mathbf{e} + \mathbf{z}$
Parameter: noise rate μ defined in Section 3.3 and (g, t, n, r) of LDPC-PRC
Output: $\mathbf{x}'' = \mathbf{x} + \mathbf{e}'$ s.t. $\mathsf{Decode}(1^\lambda, \mathsf{sk}, \mathbf{x}'') = \perp$ and $w_H(\mathbf{e}') = \mu n$

$\mathbf{x}' \leftarrow \mathbf{x} + \mathbf{z}$
$\mathbf{e} \leftarrow \mathsf{ISD}(\mathbf{G}, \mathbf{x}')$ ▷ recover $\mathbf{e}$ from $\mathbf{x}', \mathbf{G}$ with an ISD algorithm
$\mathscr{S} \leftarrow \{1, \cdots, n\} \backslash \mathsf{supp}(\mathbf{e})$ ▷ where $\mathsf{supp}(\mathbf{e})$ denotes the support set of $\mathbf{e}$
Choose $\mathscr{T} \subset \mathscr{S}$ and $|\mathscr{T}| = \mu n$
Construct $\mathbf{e}'$ with support set $\mathscr{T}$
$\mathbf{x}'' \leftarrow \mathbf{x} + \mathbf{e}'$
return $\mathbf{x}''$

Given an encoded vector $\mathbf{x} = \mathbf{Gs} + \mathbf{e} + \mathbf{z}$ and public key $\mathsf{pk} = (\mathbf{G}, \mathbf{z})$, the attacker's goal is to first recover the noise vector $\mathbf{e}$, then construct a malicious noise vector $\mathbf{e}'$ such that $\mathbf{x}+\mathbf{e}'$ cannot be successfully decoded. Since [7] does not specify an explicit noise rate bound, our objective is to maximize the resulting noise rate by adding $\mathbf{e}'$ with a *fixed* noise rate. We assume the decoding process fails once the final noise rate exceeds the designed threshold. Note that such a malicious noise attack is non-trivial because, under the same fixed noise rate, a PRC-based codeword with additional random noise of identical weight can be successfully decoded with high probability, while our malicious noise vector causes decoding to fail. We now detail the two steps of the attack.

- **Step I: Noise Recovery.** Upon receiving $\mathbf{x}$, the adversary first computes $\mathbf{x}' = \mathbf{x} + \mathbf{z} = \mathbf{Gs} + \mathbf{e}$. Let $\mathbf{H} \in \mathbb{F}_2^{(n-g)\times n}$ be the dual parity-check matrix of $\mathbf{G} \in \mathbb{F}_2^{n\times g}$ satisfying $\mathbf{HG} = \mathbf{0}$. The adversary then computes $\mathbf{v} = \mathbf{Hx}' = \mathbf{HGs} + \mathbf{He} = \mathbf{He}$. At this stage, the adversary needs to find a vector $\mathbf{e} \leftarrow \mathrm{Ber}(n, \omega)$ satisfying $\mathbf{He} = \mathbf{v}$. This reduces to the classical Syndrome Decoding (SD) problem [6] and can be solved using ISD algorithms [34,10,27,5,28]. Although the exact weight of $\mathbf{e}$ is unknown, we consider the upper bound of error-correcting capability where the noise rate equals $1/2 - \varepsilon$, as calculated by the revised inequality in Lemma 1. We adopt the Prange algorithm [34] in Theorem 3 to estimate the time complexity.
- **Step II: Noise Overlay.** Recall that the received vector is $\mathbf{x}' = \mathbf{Gs} + \mathbf{e}$ where $\mathbf{e} \leftarrow \mathrm{Ber}(n, \omega)$. If we add another random $\mathbf{e}' \leftarrow \mathrm{Ber}(n, \mu)$ to $\mathbf{x}'$, where $\mathbf{e}$ and $\mathbf{e}'$ are independent, then $\Pr[e_i + e'_i = 1] = \omega + \mu - 2\omega\mu$ and $\mathbb{E}[w_H(\mathbf{e}+\mathbf{e}')] = (\omega+\mu-2\omega\mu)n$. However, once $\mathbf{e}$ is recovered, the adversary can maliciously construct a special $\mathbf{e}'$ satisfying both $w_H(\mathbf{e}') = \mu n$ and $|\mathbf{e} \cap \mathbf{e}'| = 0$, such that

$$\mathbb{E}[w_H(\mathbf{e} + \mathbf{e}')] = (\omega + \mu)n \gg (\omega + \mu - 2\omega\mu)n.$$

  We choose μ such that PRC is designed to detect an expected number of errors equal to $(\omega + \mu - 2\omega\mu)n$, but the final noise vector $\mathbf{e} + \mathbf{e}'$ contains $(\omega+\mu)n$ errors, significantly exceeding PRC's intended threshold. Therefore, introducing $\mathbf{e}'$ successfully causes decoding failures.

Theorem 3 (Complexity of Attack-III)**.** *Given a public key* pk *and an encoded vector* $\mathbf{x}$*, there exists an adversary that can construct* $\mathbf{x}'' = \mathcal{E}(\mathbf{x})$ *to achieve* $\mathsf{Decode}(1^\lambda, \mathsf{sk}, \mathbf{x}'') \neq \mathsf{m}$ *with time complexity*

$$\mathrm{T}_{\mathsf{overlay}} = \left(\frac{1}{2} + \varepsilon\right)^{-g} \cdot n^3. \tag{5}$$

Proof. For an SD problem $\mathbf{He} = \mathbf{v}$, we first randomly shuffle the columns of $\mathbf{H}$. We then calculate the probability that this shuffle concentrates all nonzero elements of $\mathbf{e}$ in the first $n-g$ positions, leaving the last g positions as zeros (i.e., $w_H(\mathbf{e}_{n-g+1:n}) = 0$). Since each element of $\mathbf{e}$ equals 1 with probability $1/2 - \varepsilon$, this probability is

$$\mathrm{P}_{\mathsf{nz}} = \left(\frac{1}{2} + \varepsilon\right)^{g}.$$

If this condition is satisfied, we can solve the SD problem using Gaussian elimination with n^3 operations. Thus, the total time complexity for the Prange algorithm [34] is $\mathrm{T}_{\mathsf{Prange}} = \mathrm{P}_{\mathsf{nz}}^{-1} n^3$. Since the remaining steps of adding $\mathbf{e}'$ require only $O(n) \ll \mathrm{T}_{\mathsf{Prange}}$ time, the total complexity is dominated by noise recovery

$$\mathrm{T}_{\mathsf{overlay}} = \left(\frac{1}{2} + \varepsilon\right)^{-g} n^3.$$

□

Remark. In GIM, we note that the weight of added noise $\eta = 1 - 2^{-\lambda/g^2}$ is much lower than the theoretical bound $1/2 - \varepsilon$. Therefore, we also estimate $\mathrm{T}'_{\mathsf{overlay}} = (1-\eta)^{-g} \cdot n^3 = 2^{\lambda/g} \cdot n^3$ for GIM. Let $\mathrm{T}'_{\mathsf{overlay}}$ denote the cost of Attack-III with the concrete noise rate η instead of $1/2 - \varepsilon$.

4 Concrete Time Complexity Analysis

As shown in the first two rows of Table 2, though Π_{LLM} [7] and Π_{GIM} [15] have established certain parameter ranges and their interdependencies, some critical parameter settings remain unspecified, e.g., n and λ in Π_{LLM}. Therefore, we determine two parameter configurations, Configuration 1 and Configuration 2, for Π_{LLM} and Π_{GIM}, respectively. These specific parameters facilitate a concrete time complexity analysis of the proposed attacks. Note that we call Algorithm 4 and Algorithm 5 to transform the texts and images into the corresponding codewords before performing our three attacks. Since the transformation complexity is much lower than that of the attacks, we only take the latter into account.

4.1 Parameters Configurations for Π_{LLM}

Configuration 1 (Π_{LLM})**.** *For* Π_{LLM}*, we set* $n \leq 2^{17}$*,* $r \leq 0.99n$*,* $g = \lambda = \log \binom{n}{t}$*, and* $3 \leq t \leq 14$*.*

Table 2: Parameters of LDPC-PRC for Π_{LLM} and Π_{GIM}.

	n	r	g	λ	t
Π_{LLM} [7]	-	$0.99n$	$c\log^2 n$	–	$\log n$
Π_{GIM} [15]	2^{14}	$n-k-\lambda$	$\log\binom{n}{t}$	g	$3 \le t \le 7$
Configuration I (for Π_{LLM})	2^{17}	$0.99n$	$\log\binom{n}{t}$	g	$3 \le t \le 14$
Configuration II (for Π_{GIM})	2^{14}	$n-k-\lambda$	$\log\binom{n}{t}$	g	$3 \le t \le 7$

When analyzing the computational complexity of attacks against Π_{LLM}, we use the maximum parameter values in Configuration 1 to provide the upper bound of security: $n = 2^{17}, r = \lfloor 0.99n \rfloor$, and $g = \lambda = \binom{n}{t}$ for each t. Next, we introduce the rationales for parameter setting in Configuration 1 as follows:

- **The code length** n: According to Construction 2, each token is mapped into binary bit strings. We assume the length of binary bit strings is smaller than 32 bits, i.e., the size of the token alphabet[4] is smaller than 2^{32}. Meanwhile, we assume that the max output length is $2^{12} = 4,096$ tokens, which is a common setting for LLMs (e.g., gpt-3.5-turbo [29] and gpt-4-turbo [30]).[5] Note that in practical scenarios, the output length of LLMs typically does not reach the maximum limit. If the defender requires stronger robustness against the binary deletion channel (BDC) as described in [7], the PRC must be embedded into a longer code. Consequently, the length of PRC becomes significantly shorter than the output length of LLMs. As a result, it is natural and reasonable to assume that the code length n is not larger than $32 \times 2^{12} = 2^{17}$.
- **The column size** g **and security parameter** λ: Though [7] sets $g = c\log^2 n$ where c is a constant, but no concrete value of c and expression of λ is given. Hence, we follow [15] to set $g = \lambda = \log\binom{n}{t}$.
- **The Row-wise nonzero count** t: We set $3 \le t \le 14$ for Π_{LLM} due to the following reasons. When t is 1 or 2, the complexity of Attack-I is linear in n. Therefore, to exclude this trivial attack, we restrict our analysis to $t \ge 3$. Let $\rho = 1/2 - \varepsilon$ be the maximum error-correcting capability. Since no explicit value of ρ is discussed in [7], we use the experimental results in Π_{GIM} as reference. During the experiments in Section 5.3, we find that Π_{GIM} has to correct vectors with noise rate $\rho = 0.074$. Hence, we bound $\rho \ge 0.074$, i.e.,

[4] The size of the alphabet varies from thousands to millions; thus, 2^{32} is a very loose upper bound.

[5] We also notice that the recently released, more advanced LLMs are capable of supporting $32,768$ max output tokens, e.g., gpt-4.1-turbo [31]. We will prove that Π_{LLM} can still not reach an 128-bit security even with such length. More details are available in Section 6.1.

$\varepsilon \leq 0.426$. However, when we try to calculate the upper bound of t with r and ε following [7], we notice a mistake in their calculation, i.e.,

$$(2\varepsilon)^t/2 > r^{-1/4} \nRightarrow t < 1 + \log(r)/4\log(1/2\varepsilon).$$

Therefore, we give a corrected version in Lemma 1 and get $t \leq 14$.

Lemma 1. *In order to decode successfully with high probability, we need* $(2\varepsilon)^t/2 > r^{-1/4}$*, i.e.,*

$$t < \frac{\frac{1}{4}\log r - 1}{\log(1/2\varepsilon)}.$$

Proof. With $(2\varepsilon)^t/2 > r^{-1/4}$, we have $(2\varepsilon)^t > 2r^{-1/4}$. Since $0 < \varepsilon < 1/2$, we can simplify the inequality to $t < \log_{2\varepsilon}(2r^{-1/4})$, i.e., $t < \frac{1-1/4\log r}{\log(2\varepsilon)} = \frac{\frac{1}{4}\log r - 1}{\log(1/2\varepsilon)}$. □

4.2 Parameters Configurations for $\Pi_{\mathbf{GIM}}$

Configuration 2 (Π_{GIM})**.** *For* Π_{GIM}*, we set* $n = 2^{14}$*,* $g = \lambda = \binom{n}{t}$*,* $k = \lambda +$ *message_bit* $+$ *parity_bit where message_bit* $= 512$*, and parity_bit* $= \log_2(\mathrm{FPR})$ *where* $\mathrm{FPR} = 10^{-5}$*. When calculating* $\mathbf{y} = \mathbf{Gs} + \mathbf{e}$*, the added noise vector* $\mathbf{e}$ *has error weight* $\eta = 1 - 2^{-\lambda/g^2}$*.*

For Configuration 2, we employ the default parameters from [15]. Note that the weight t is set to 3 in most cases discussed in [15], and it is also suggested to set $t = \log(n)/2 = 7$ to ensure cryptographic undetectability. For a complete analysis of our attacks, we vary the weight t from 3 to 7 and change r, g, k, λ, and η accordingly.

4.3 Overall Performance

We present a comparison between the complexity of our attacks and the claimed security levels in Table 3 and Table 4, and illustrate the trend of complexity with varying parameters in Figure 1.

Performance on Π_{LLM}. First, as shown in Table 3, the complexities of the three attacks are lower than the claimed security level λ across all parameters. Second, Figure 1a presents clearer trends in the complexities. For the first two attacks, their respective attack complexities (i.e., $\mathrm{T}_{\mathsf{partial}}$, and $\mathrm{T}_{\mathsf{dis}}$) exhibit a consistent increasing trend with growing t. Nevertheless, it consistently remains below 128 bits, demonstrating that our attacks remain feasibly threatening. Notably, for Attack-II, the frequency of weak keys is non-negligible and approaches 100% for parameters $t = 3$ and $t = 4$, making the attack almost always effective in these cases. For Attack-III, we can observe that even as t increases, the value of $\mathrm{T}_{\mathsf{overlay}}$ remains nearly unchanged and consistently stays around 73 bits, indicating that Π_{LLM} exhibits vulnerability to our noise overlay attack across a reasonable range of parameters.

Table 3: Time complexities of all attacks on Π_{LLM}.

t	ε	ρ	$\log_2^{\mathrm{T}_{\mathsf{partial}}}$	$\log_2^{\mathrm{P}_{\mathsf{weak}}^{\mathrm{LLM}}}$	$\log_2^{\mathrm{T}_{\mathsf{dis}}}$	$\log_2^{\mathrm{T}_{\mathsf{overlay}}}$	λ
3	0.236	0.264	21.30	−0.00	22.58	72.21	48
4	0.285	0.215	29.19	−0.00	22.98	73.02	63
5	0.319	0.181	36.84	−3.91	27.20	73.49	78
6	0.344	0.156	44.28	−11.89	35.42	73.57	92
7	0.363	0.137	51.59	−19.66	43.39	73.61	106
8	0.377	0.123	58.76	−27.20	51.11	73.64	120
9	0.389	0.111	65.84	−34.55	58.62	73.66	134
10	0.399	0.101	72.82	−41.73	65.94	73.67	148
11	0.408	0.092	79.71	−48.75	73.08	73.54	161
12	0.415	0.085	86.54	−55.64	80.09	73.56	175
13	0.421	0.079	93.29	−62.40	86.95	73.46	188
14	0.426	0.074	99.99	−69.04	93.69	73.37	201

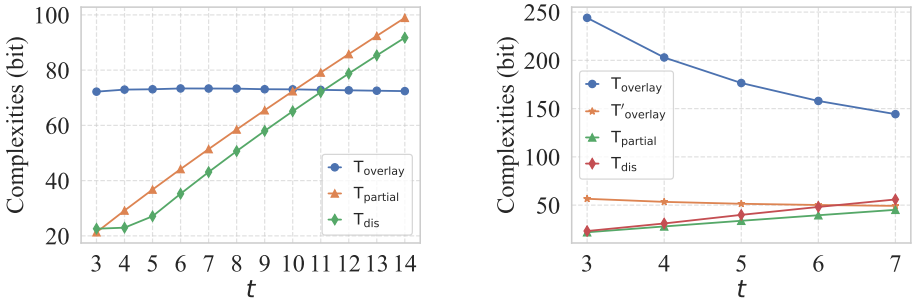
Table 4: Time complexities of all attacks on Π_{GIM}.

t	ε	ρ	η	$\log_2^{\mathrm{T}_{\mathsf{partial}}}$	$\log_2^{\mathrm{P}_{\mathsf{weak}}^{\mathrm{GIM}}}$	$\log_2^{\mathrm{T}_{\mathsf{dis}}}$	$\log_2^{\mathrm{T}_{\mathsf{overlay}}}$	$\log_2^{\mathrm{T}'_{\mathsf{overlay}}}$	λ
3	0.282	0.218	0.018	21.88	−0.01	23.16	244.03	56.56	39
4	0.325	0.175	0.013	27.91	−7.83	31.01	202.96	53.37	51
5	0.354	0.146	0.011	33.78	−16.71	39.92	176.51	51.40	63
6	0.375	0.125	0.009	39.51	−24.87	48.11	157.93	50.15	74
7	0.391	0.109	0.008	45.14	−32.60	55.87	144.27	49.22	85

Performance on Π_{GIM}. Similarly, Table 4 shows that the complexities of our three attacks remain below the claimed security level λ for all parameter settings. The corresponding trends are visualized in Figure 1b. The first two attacks exhibit increasing complexity with growing t, which is similar to the performance on Π_{LLM}. Although $\mathrm{P}_{\mathsf{weak}}^{\mathrm{GIM}}$ of GIMs is larger than that of LLMs, it remains high under concrete parameters. When $t = 3$, weak keys occur with nearly 100%, indicating practical vulnerability. For Attack-III, we can observe that both the complexity $\mathrm{T}_{\mathsf{overlay}}$ and $\mathrm{T}'_{\mathsf{overlay}}$ decrease as t increases, which comes from the decrease in the theoretical noise rate ρ and the concrete noise rate η. Due to the gap between ρ and η , there is a huge difference between $\mathrm{T}_{\mathsf{overlay}}$ and $\mathrm{T}'_{\mathsf{overlay}}$. Taking into account $\mathrm{T}'_{\mathsf{overlay}}$ instead of $\mathrm{T}_{\mathsf{overlay}}$, all complexities are below 64 bits, indicating practical threats under the parameter choices of Π_{GIM}.

4.4 Further Analysis of Watermark Removal Attacks

Performance on Π_{LLM}. The experimental results show that $\mathrm{T}_{\mathsf{overlay}}$ increases slightly as t increases when $3 \leq t \leq 12$, but decreases when $t \geq 12$. This phenomenon indicates that a larger t does not always lead to a higher security level. We also estimate all reasonable parameters on $t > 14$, which concretely



(a) LLM. (b) GIM.

Fig. 1: Time complexities of all attacks. The complexities are expressed in logarithm form.

verifies this conclusion. We further analyze this phenomenon on the basis of the following truth.

- $\mathrm{T}_{\mathsf{overlay}} \propto (1-\rho)^{-g} = (\frac{1}{2}+\varepsilon)^{-g}$.
- g increases as t increases since $g = \log \binom{n}{t}$.
- ε increases as t increases because of the constraint in Lemma 1.

Therefore, for small values of t, the term g dominates the behavior of $\mathrm{T}_{\mathsf{overlay}}$, causing $\mathrm{T}_{\mathsf{overlay}}$ to increase as t increases. As t progressively increases, ε gradually becomes dominant, leading to a decrease in $\mathrm{T}_{\mathsf{overlay}}$. Second, although the estimated $\mathrm{T}_{\mathsf{overlay}}$ seems large for small t compared to the other attacks, the defender might add a much lower noise rate as Π_{LLM} does, thereby causing a much lower $\mathrm{T}_{\mathsf{overlay}}$.

Performance on Π_{GIM}. $\mathrm{T}_{\mathsf{overlay}}$ decreases sharply when $3 \leq t \leq 7$ since the error weight dominates the complexity. However, the actual error weight $\eta << \rho$ in all cases, making $\mathrm{T}'_{\mathsf{overlay}}$ much lower than the theoretical bound $\mathrm{T}_{\mathsf{overlay}}$. The gap could be as large as 187 bits, indicating the practical threat of our attack.

5 Real-World Evaluations

To further demonstrate the practical threat posed by our attacks, we launch Attack-I and Attack-II against real-world large generative models.

5.1 Setups

Target LLM. We employ DeepSeek-R1-Distill-Qwen-7B [16] as our real-world target LLM. Each response is generated with a fixed length of $1,024$ tokens, chosen from a vocabulary of size 152,064. To embed watermarks, we apply Π_{LLM} [7] as described in Section 2.3. Following Configuration 1, we set $n = \lceil \log_2(152,064) \rceil * 1,024 = 18,432$, $r = \lfloor 0.95n \rfloor = 17,510$, and t as 3 or 4.

Target GIM. We adopt stable-diffusion-2-1-base [36] as our target GIM. To generate images, we use the Multistep DPMSolver Scheduler [25] as the de-noising scheduler $f(\cdot)$ of Algorithm 2 and set the de-noising step s to 50. We apply the Π_{GIM} scheme to embed watermarks with $t = 3$ or 4, and all other parameters are set as in Configuration 2. During watermark verification, we perform DDIM Inversion [39] as the noise inversion scheduler $z(\cdot)$ of Algorithm 5 with the same number of steps and guidance scale as the image generation process.

AI-Generated Contents. In each set of parameters, we randomly generate 128 pairs of $\mathsf{pk} = (\mathbf{G}, \mathbf{z})$. For each pk, we generate 16 texts or images. Meanwhile, we also generate non-watermarked content with the same number as a control group. The examples of LLM prompts are presented in Appendix B. For GIM, we follow the model hyperparameter settings in [15] and sample the prompts from the test split of the Gustavosta/Stable-Diffusion-Prompts dataset [37]. A subset of the generated text and image samples is provided in Appendix C.

Settings of Attack-I and Attack-II. When attacking LLMs, we set the temperature to 1.8, and the parameters for attacking are set as follows: for $t = 3$, we set $|\mathscr{L}_1'| = |\mathscr{L}_2'| = 18,432$ and $\tau = 0.60$ for both Attack-I and Attack-II. When $t = 4$, we set $|\mathscr{L}_1'| = |\mathscr{L}_2'| = 5 \times 10^6$, and $\tau = 0.55$ for Attack-I and $\tau = 0.65$ for Attack-II. The parameter settings for attacking GIMs are as follows: When $t = 3$, we set $|\mathscr{L}_1'| = |\mathscr{L}_2'| = 16,384$ and $\tau = 0.75$ for both Attack-I and Attack-II. When $t = 4$, we set $|\mathscr{L}_1'| = |\mathscr{L}_2'| = 5 \times 10^5$ and $\tau = 0.75$ for both Attack-I and Attack-II.

Evaluation Metrics. (1) For Attack-I, we define the success rate as the probability that $|\mathscr{L}'| > 0$. For Attack-II, we first identify weak keys and then perform our attack on them. The success rate of Attack-II corresponds to the probability of encountering weak keys. (2) We also evaluate the true positive rate (TPR) and false positive rate (FPR) for watermark detection upon the successful attacks. Let TPR_0 denote the TPR when testing the original codeword generated by the defender, i.e., $\mathbf{x} = \mathbf{G}\mathbf{s} + \mathbf{e} + \mathbf{z}$, and FPR_0 denote the FPR when testing a uniformly random vector. Meanwhile, we use TPR_1 to denote the TPR for the extracted codeword from the watermarked content, which includes both the defender's added noise and the noise introduced by the schemes. Specifically, the additional noise of Π_{LLM} is introduced in Line 5 and Line 7 of Algorithm 3, while the noise of Π_{GIM} arises from the error of DDIM inversion. Correspondingly, FPR_1 represents the FPR for the codeword corresponding to the unwatermarked content.

5.2 Performance on LLMs

Impracticality of Π_{LLM}. Prior to an in-depth examination of the attack effectiveness, it is imperative to first elucidate the impracticality of Π_{LLM}. Recall in Algorithm 3, the tokens are biased according to the probability p' derived from the codewords, which means the bit accuracy of the recovered message depends on the average token entropy of the LLM output. To satisfy LDPC-PRC 's correction bounds, the LLM must produce high-entropy token distributions, which

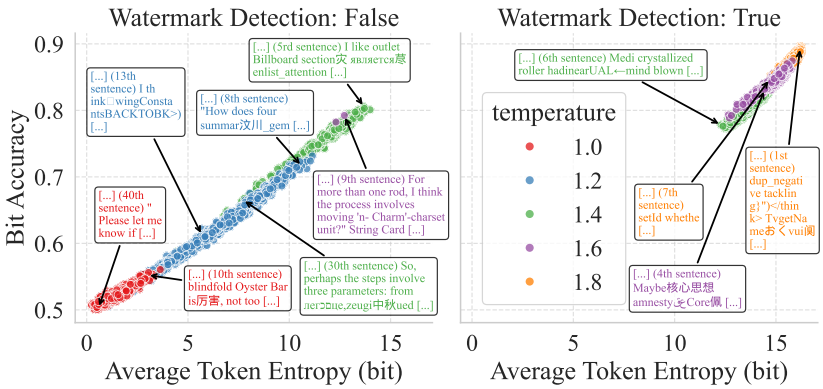


Fig. 2: The relationship between the readability of generated text and the detectability of watermarks. We set $t = 3$ for Π_{LLM}.

only occur with low confidence in next-token prediction. In other words, Π_{LLM} can only embed a detectable watermark in low-quality responses.

To examine this, we generate watermarked texts using temperature values temp from 1.0 to 1.8, controlling output entropy. As shown in Figure 2, the embedded watermark is undetectable at temperatures of 1.0 and 1.2. When $\mathsf{temp} = 1.4$, watermarks are detectable in 60% of the generated outputs, yet the textual readability has been severely compromised. As illustrated by the green texts of Figure 2, the LLM's outputs mix multiple languages. With further increases in the value of t to 1.6 and 1.8, the watermark detection rate shows a corresponding enhancement; however, the text becomes almost unreadable (represented by purple and orange texts in Figure 2). Owing to the above factors, we launch attacks against texts capable of carrying watermarks, disregarding their readability.

Performance of Attack-I. As described in Table 5, regardless of whether $t = 3$ or $t = 4$, Attack-I maintains a perfect success rate of 100%, which means we can stably construct $\mathscr{L}'$ which satisfies $|\mathscr{L}'| > 0$. More precisely, we can recover an average of 8.8 rows of the secret keys. Meanwhile, the attainment of 100% TPR_0 alongside 0% FPR_0 confirms that Attack-I possesses a requisite precision to discriminate between PRC codewords and random vectors. However, the TPR_1 on the real-world output texts becomes 61% when $t = 3$ and 98% when $t = 4$. This degradation is attributable to the additional noise introduced during the token sampling procedure in Algorithm 4. Specifically, the error rate increases from 0.10 for the original codeword to 0.20 for the recovered codeword.

Performance of Attack-II. On the other hand, Attack-II achieves complete success at $t = 3$, yet drops to 63% when $t = 4$. It demonstrates a considerable likelihood of weak keys occurring within LLM scenarios. Furthermore, the high TPR_0 values and low FPR_0 values collectively demonstrate Attack-II's efficacy in differentiating between PRC codewords and random vectors, demonstrating

Table 5: Watermark detection attacks against real-world LLM.

Attack	t	TPR_0	FPR_0	TPR_1	FPR_1	Success Rate
I	3	100%	0%	61%	0%	100%
	4	100%	0%	98%	0%	100%
II	3	100%	0%	100%	0%	100%
	4	98%	10%	59%	10%	63%

Table 6: Watermark detection attacks against real-world GIM.

Attack	t	TPR_0	FPR_0	TPR_1	FPR_1	Success Rate
I	3	99%	0%	93%	0%	100%
	4	93%	4%	66%	4%	78.9%
II	3	100%	0%	96%	0%	100%
	4	100%	0%	100%	0%	2.3%

that with up to 22.09 duplicated rows on average, the distinguisher performs very well. Unlike Attack-I, the TPR_1 sustains a perfect 100% on real-world texts at $t = 3$. When $t = 4$, the gap between TPR_1 and FPR_1 remains sufficiently large to enable reliable detection.

5.3 Performance on GIMs

Adversary's Capability. Given an image, for both defenders verifying watermarks and adversaries launching detection attacks, it is necessary to utilize Algorithm 5 to recover the codeword from the image. There are two neural networks, `U-net` and `E-net`, used in Algorithm 5. The defender typically employs the same `U-net` utilized in the generation process, along with an `E-net` compatible with the `D-net` that was used for generating the image, for a highly precise codeword extraction. For the adversary, we first assume that they can obtain the same `U-net` and `E-net` as the defender. However, considering that model parameters are typically kept confidential as commercial secrets, we also consider that the adversary will employ a proxy model that differs from the defender's.

Attack Performance Under the Same Neural Networks. Our attack performance on Π_{GIM} is shown in Table 6. In our experiments, we find the error rate of the original codeword is 0.018 and the noise rate of the received codeword is 0.074. With a lower-weight noise vector compared to Π_{LLM}, we achieve more effective attacks. When $t = 3$, although only 5.8 rows are recovered on average (vs. 8.8 in Π_{LLM}) in Attack-I, the TPR_1 is 93% (vs. 61% in Π_{LLM}) with FPR remaining 0%. For Attack-II, the weak key frequency is 100% and duplicated rows average 14.86. Our distinguisher also performs well with $\text{TPR}_1 = 96\%$ and $\text{FPR}_1 = 0\%$. The cases of $t = 4$ are similar to $t = 3$.

Table 7: Watermark detection attacks against real-world GIM with different proxy models when $t = 3$.

Attack	TPR_0	$\text{TPR}_{\mathsf{SD21}}$	$\text{TPR}_{\mathsf{SD15}}$	$\text{TPR}_{\mathsf{SD20}}$	FPR	Success Rate
I	98%	95%	61%	95%	0%	100%
II	100%	100%	95%	100%	0%	100%

Attack Performance Using Proxy Neural Networks. To verify that our attacks also work without access to the target model's neural network in real-world scenarios, we conduct extra experiments with other proxy models. Specifically, we generate watermarked images with stable-diffusion-2-1-base, but extract the codewords through Algorithm 5 with the autoencoders (`E-net`,`D-net`) from stable-diffusion-2-1-base (SD21), stable-diffusion-v1-5 (SD15) and stable-diffusion-2-base (SD20) [36]. Then, we calculate TPR values of $\text{TPR}_{\mathsf{SD21}}$, $\text{TPR}_{\mathsf{SD15}}$, and $\text{TPR}_{\mathsf{SD20}}$. The parameter choice is the same as the formal set except that $\tau = 0.60$ for both Attack-I and Attack-II.

The results are presented in Table 7. We display only one FPR column, as the FPR is 0% for all vectors in the experiments. Our attack with proxy neural networks remains effective in both Attack-I and Attack-II. In particular, we observe that SD20 performs almost as well as the original target model SD21. These experimental results further demonstrate the practical threats posed by our proposed watermark removal attacks.

6 Mitigations

6.1 Parameter Suggestions

In this part, we discuss how to mitigate Attack-I and Attack-III through different parameter selections, while Attack II can be avoided by modifying the KeyGen() function.

We first estimate the security of Π_{LLM} with different choices of n, t, while keeping $g = \log \binom{n}{t}$ and $r = 0.99n$. To defend Attack-I, the defender should increase t or n. For example, to achieve a cryptographic security ($\lambda \geq 128$), for $t = 9, 11, 13$, and 15, we suggest $n > 2^{32}, 2^{26}, 2^{23}$, and 2^{20}, respectively. It is noteworthy that increasing t also reduces the error-correcting ability. Therefore, a careful trade-off is necessary. As for Attack-III, the most efficient defense method is to increase n rather than t. Specifically, when $n = 2^{24}$, we find $\text{T}_{\mathsf{overlay}} \leq 2^{122.21}$ for all choices of t. Therefore, we suggest $n > 2^{24}$ for Π_{LLM} to achieve a 128-bit security level. However, for the current state-of-the-art LLMs such as gpt-4.1-turbo [31], the maximum token length is $2^{15} = 32,768$, i.e., $n = 2^{20}$. As a result, constructing Π_{LLM} satisfying 128-bit security remains challenging in practice.

The above parameter suggestions for LLMs also apply to GIMs. However, as discussed in [36], the maximum dimension of the initial latent is $4 \times 128 \times 128$, which corresponds to $n = 2^{16}$, still leaving a large gap from cryptographic security. Thus, achieving the 128-bit security level for Π_{GIM} is also impractical.

Algorithm 10 Revised Key Generation Algorithm

Input: Parameters: n, $r = 0.99n$, g, t
Output: Matrices $\mathbf{P}$ and $\mathbf{G}$ for watermark generation

1: Initialize an empty list for rows of $\mathbf{P}$
2: **for all** $i \in [0.99n]$ **do**
3: Sample a random t-sparse vector $\mathbf{s}_i \in \mathbb{F}_2^n$
4: Append $\mathbf{s}_i$ to the list of rows for $\mathbf{P}$
5: **end for**
6: Let $\mathbf{P}$ be the matrix whose rows are $\mathbf{s}_1, \ldots, \mathbf{s}_{0.99n}$
7: **Perform Gaussian elimination on P and transform it to system form** $[\mathbf{I}_r || \mathbf{P}_1]$
8: **Let** $\mathbf{G}_0 = [\mathbf{P}_1^T || \mathbf{I}_{n-r}]^T$
9: **Random select** g **columns from** $\mathbf{G}_0$ **to construct** $\mathbf{G} \in \mathbb{F}_2^{n \times g}$
10: Sample a random permutation $\mathbf{\Pi} \in \mathbb{F}_2^{n \times n}$, and let $\mathbf{P} \leftarrow \mathbf{P}\mathbf{\Pi}^{-1}, \mathbf{G} \leftarrow \mathbf{\Pi}\mathbf{G}$
11: **return** $(\mathbf{P}, \mathbf{G})$

6.2 Implementation Suggestions for GIMs

As discussed in [7], the condition $t = \Omega(\log(n))$ is necessary to guarantee security. However, Π_{GIM} [15] adds extra parity bits and uses the belief propagation algorithm to recover the message, which requires t to be a constant. When t decreases from $\Omega(\log(n))$ to a constant c, the row-wise entropy of the secret key space $\binom{r}{t}$ also degrades to $\Omega(r^c) = \Omega(n^c)$. Consequently, a brute-force attack by enumerating all possible rows of the secret key succeeds with polynomial-time complexity $O(n^c)$. Therefore, we suggest removing the $\mathsf{Decode}'()$ procedure, enabling a larger t in parameter choice.

Moreover, as discussed in Section 4.4, Π_{GIM} adds a noise vector with weight $\eta \ll \rho$, which results in an extremely efficient concrete attack compared to the theoretical complexity $\mathrm{T}_{\mathsf{overlay}}$. In fact, since $\lambda = g = \Theta(\log^2 n)$ and k could be seen as the sum of g and a constant c in Π_{GIM}, the complexity of Attack-III, $\mathrm{T}'_{\mathsf{overlay}} = 2^{k/g} n^3 = 2^{1+c/\Theta(\log^2 n)} n^3 = O(n^3)$, is only *polynomial*. Therefore, we suggest increasing the weight of the added noise vector.

6.3 Revision of Key Generation Algorithm

To defend against Attack-II, we remove the acceleration trick in Algorithm 6 to avoid the weak key with high frequency and construct a revised version in Algorithm 10. We highlight the revised procedure in **green**. Specifically, we first randomly sample the secret key, and then use Gaussian elimination to generate the public key $\mathbf{G}$. It is obvious that the revised algorithm satisfies the random assumption of $\mathbf{P}$ and $\mathbf{G}$. Note that a concomitant drawback of this mitigation is its impact on computational efficiency, i.e., the complexity of the revised algorithm increases from $O(n^2)$ to $O(n^3)$ due to the Gaussian elimination in Line 7. To preserve the randomness and independence of the rows of $\mathbf{G}$, this increase in time complexity is a necessary trade-off.

7 Conclusion

This paper presents the first cryptanalysis of PRC by proposing three attacks, including two distinguishing attacks and one noise overlay attack. Through rigorous theoretical analysis of success rates and complexity, we demonstrate that the current instantiation LDPC-PRC exhibits security vulnerabilities. Our attack complexity is lower than its claimed security guarantees, with most attacks requiring fewer than 128 bits of complexity. Furthermore, we implement PRC-based watermarking schemes in real-world large generative models to validate the practical threat. We also propose three defenses, including parameter and implementation recommendations, along with a revised key generation algorithm. Beyond security analysis, we refine PRC's parameter ranges and prove that even without attacks, the current PRC scheme faces challenges in practical LLM applications. Our findings will facilitate the development of future AIGC watermarking schemes that achieve robustness, undetectability, and practicality.

Future Work. Several open problems remain for future work. First, as discussed in Section 5.2, Π_{LLM} is not directly applicable to real-world scenarios. Since our work focuses on security analysis, we have not yet provided concrete measures to enable the practical deployment of PRC in large-scale LLMs. Second, for diffusion models, the image-to-latent inversion relies on neural network outputs rather than a closed-form solution, preventing a theoretical characterization of the resulting uncertainty in attack performance. Finally, regarding parameters (n, g, t, r), because r is typically set slightly below n with negligible impact on security or error correction, we focus on n and t in our analysis of practical schemes. While we show that increasing g to k in Π_{GIM} yields no asymptotic advantage, the precise discussion of g remains open for further investigation.

References

1. Aaronson, S.: My ai safety lecture for ut effective altruism (2022), https://scottaaronson.blog/?p=6823
2. Al-Busaidi, A.S., Raman, R., Hughes, L., Albashrawi, M.A., Malik, T., Dwivedi, Y.K., Al- Alawi, T., AlRizeiqi, M., Davies, G., Fenwick, M., Gupta, P., Gurpur, S., Hooda, A., Jurcys, P., Lim, D., Lucchi, N., Misra, T., Raman, R., Shirish, A., Walton, P.: Redefining boundaries in innovation and knowledge domains: Investigating the impact of generative artificial intelligence on copyright and intellectual property rights. Journal of Innovation & Knowledge **9**(4), 100630 (2024)
3. Alrabiah, O., Ananth, P., Christ, M., Dodis, Y., Gunn, S.: Ideal pseudorandom codes. arXiv preprint arXiv:2411.05947 (2024)
4. An, B., Ding, M., Rabbani, T., Agrawal, A., Xu, Y., Deng, C., Zhu, S., Mohamed, A., Wen, Y., Goldstein, T., Huang, F.: WAVES: benchmarking the robustness of image watermarks. In: Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024. OpenReview.net (2024)
5. Becker, A., Joux, A., May, A., Meurer, A.: Decoding random binary linear codes in $2^{n/20}$: How $1 + 1 = 0$ improves information set decoding. In: Advances in Cryptology–EUROCRYPT 2012: 31st Annual International Conference on the

Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings 31. pp. 520–536. Springer (2012)
6. Berlekamp, E., McEliece, R., Van Tilborg, H.: On the inherent intractability of certain coding problems (corresp.). IEEE Transactions on Information theory **24**(3), 384–386 (2003)
7. Christ, M., Gunn, S.: Pseudorandom error-correcting codes. In: Reyzin, L., Stebila, D. (eds.) Advances in Cryptology – CRYPTO 2024. Lecture Notes in Computer Science, vol. 14925, pp. 325–347. Springer, Santa Barbara, CA, USA (2024)
8. Christ, M., Gunn, S., Zamir, O.: Undetectable watermarks for language models. In: Agrawal, S., Roth, A. (eds.) The Thirty Seventh Annual Conference on Learning Theory, June 30 - July 3, 2023, Edmonton, Canada. Proceedings of Machine Learning Research, vol. 247, pp. 1125–1139. PMLR (2024)
9. Deb, K., Al-Seraj, M.S., Hoque, M.M., Sarkar, M.I.H.: Combined dwt-dct based digital image watermarking technique for copyright protection. In: 2012 7th International Conference on Electrical and Computer Engineering. pp. 458–461. IEEE (2012)
10. Dumer, I.: On minimum distance decoding of linear codes. In: Proc. 5th Joint Soviet-Swedish Int. Workshop Inform. Theory. pp. 50–52. Moscow (1991)
11. Fairoze, J., Garg, S., Jha, S., Mahloujifar, S., Mahmoody, M., Wang, M.: Publicly-detectable watermarking for language models. IACR Communications in Cryptology **1**(4) (2025)
12. Fernandez, P., Couairon, G., Jégou, H., Douze, M., Furon, T.: The stable signature: Rooting watermarks in latent diffusion models. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 22466–22477 (2023)
13. Garg, S., Gunn, S., Wang, M.: Black-box crypto is useless for pseudorandom codes (2025), https://arxiv.org/abs/2506.01854
14. Ghentiyala, S., Guruswami, V.: New constructions of pseudorandom codes. arXiv preprint arXiv:2409.07580 (2024)
15. Gunn, S., Zhao, X., Song, D.: An undetectable watermark for generative image models. In: The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025. OpenReview.net (2025)
16. Guo, D., Yang, D., Zhang, H., Song, J., Zhang, R., Xu, R., Zhu, Q., Ma, S., Wang, P., Bi, X., et al.: Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning (2025)
17. Gupta, M.: Ai in social media: Statistics, trends driving content creation, https://colorwhistle.com/ai-in-social-media/
18. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (6 2016)
19. Jovanović, N., Staab, R., Vechev, M.: Watermark stealing in large language models. In: ICML 2024 (2024)
20. Kirchenbauer, J., Geiping, J., Wen, Y., Katz, J., Miers, I., Goldstein, T.: A watermark for large language models. In: International Conference on Machine Learning. pp. 17061–17084. PMLR (2023)
21. Kirchenbauer, J., Geiping, J., Wen, Y., Shu, M., Saifullah, K., Kong, K., Fernando, K., Saha, A., Goldblum, M., Goldstein, T.: On the reliability of watermarks for large language models. In: The Twelfth International Conference on Learning Representations (2024)
22. Krishna, K., Song, Y., Karpinska, M., Wieting, J., Iyyer, M.: Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense. In: Oh, A.,

Naumann, T., Globerson, A., Saenko, K., Hardt, M., Levine, S. (eds.) Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023 (2023)
23. Kuditipudi, R., Thickstun, J., Hashimoto, T., Liang, P.: Robust distortion-free watermarks for language models. Trans. Mach. Learn. Res. **2024** (2024)
24. Liu, A., Pan, L., Lu, Y., Li, J., Hu, X., Zhang, X., Wen, L., King, I., Xiong, H., Yu, P.: A survey of text watermarking in the era of large language models. ACM Computing Surveys **57**(2), 1–36 (2024)
25. Lu, C., Zhou, Y., Bao, F., Chen, J., Li, C., Zhu, J.: Dpm-solver: a fast ode solver for diffusion probabilistic model sampling in around 10 steps. In: Proceedings of the 36th International Conference on Neural Information Processing Systems. NIPS '22, Curran Associates Inc., Red Hook, NY, USA (2022)
26. Lucchi, N.: Chatgpt: A case study on copyright challenges for generative artificial intelligence systems. European Journal of Risk Regulation **15**(3), 602–624 (2024)
27. May, A., Meurer, A., Thomae, E.: Decoding random linear codes in $\tilde{\mathcal{O}}(2^{0.054n})$. In: International Conference on the Theory and Application of Cryptology and Information Security. pp. 107–124. Springer (2011)
28. May, A., Ozerov, I.: On computing nearest neighbors with applications to decoding of binary linear codes. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques. pp. 203–228. Springer (2015)
29. OpenAI: gpt-3.5-turbo (2023), https://platform.openai.com/docs/models/gpt-3.5-turbo
30. OpenAI: gpt-4-turbo (2023), https://platform.openai.com/docs/models/gpt-4-turbo
31. OpenAI: gpt-4.1-turbo (2025), https://platform.openai.com/docs/models/gpt-4.1
32. OpenAI, Hurst, A., Lerer, A., Goucher, A.P., et al.: Gpt-4o system card (2024)
33. Pan, L., Liu, A., He, Z., Gao, Z., Zhao, X., Lu, Y., Zhou, B., Liu, S., Hu, X., Wen, L., et al.: Markllm: An open-source toolkit for llm watermarking. arXiv preprint arXiv:2405.10051 (2024)
34. Prange, E.: The use of information sets in decoding cyclic codes. IRE Transactions on Information Theory **8**(5), 5–9 (1962)
35. Predis.ai: Ai ad generator, https://predis.ai/
36. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., Ommer, B.: High-resolution image synthesis with latent diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). pp. 10684–10695 (6 2022)
37. Santana, G.: Gustavosta/stable-diffusion-prompts (2022), https://huggingface.co/datasets/Gustavosta/Stable-Diffusion-Prompts
38. Sato, R., Takezawa, Y., Bao, H., Niwa, K., Yamada, M.: Embarrassingly simple text watermarks (2023)
39. Song, J., Meng, C., Ermon, S.: Denoising diffusion implicit models (2020)
40. Tallam, K., Cava, J.K., Geniesse, C., Erichson, N.B., Mahoney, M.W.: Removing watermarks with partial regeneration using semantic information (2025)
41. Tancik, M., Mildenhall, B., Ng, R.: Stegastamp: Invisible hyperlinks in physical photographs. In: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. pp. 2117–2126 (2020)
42. Wen, Y., Kirchenbauer, J., Geiping, J., Goldstein, T.: Tree-rings watermarks: Invisible fingerprints for diffusion images. Advances in Neural Information Processing Systems **36**, 58047–58063 (2023)

43. Yang, Y., Gao, R., Wang, X., Ho, T.Y., Xu, N., Xu, Q.: Mma-diffusion: Multimodal attack on diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 7737–7746 (2024)
44. Yang, Z., Zeng, K., Chen, K., Fang, H., Zhang, W., Yu, N.: Gaussian shading: Provable performance-lossless image watermarking for diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 12162–12171 (2024)
45. Zhao, X., Ananth, P.V., Li, L., Wang, Y.X.: Provable robust watermarking for AI-generated text. In: The Twelfth International Conference on Learning Representations (2024)
46. Zhao, X., Gunn, S., Christ, M., Fairoze, J., Fabrega, A., Carlini, N., Garg, S., Hong, S., Nasr, M., Tramer, F., Jha, S., Li, L., Wang, Y.X., Song, D.: SoK: Watermarking for AI-Generated Content . In: 2025 IEEE Symposium on Security and Privacy (SP). pp. 2621–2639 (2025)
47. Zhu, J., Kaplan, R., Johnson, J., Fei-Fei, L.: Hidden: Hiding data with deep networks. In: Proceedings of the European conference on computer vision (ECCV). pp. 657–672 (2018)

Appendix

A Distinguishing Attacks in Absence of Public Keys

We further propose two new attacks, Attack-IV and Attack-V, when public keys are unavailable.

A.1 Attack-IV: Low-Weight Parity Distinguisher

To justify the applicability of our algorithms in the absence of a public key, we present Attack-IV in Algorithm 11, which can be viewed as a natural generalization of Attack-I in Algorithm 7.

Algorithm 11 `Attack-IV`

Input: Target codewords $\mathscr{X} = \{\mathbf{x}^{(1)}, \cdots, \mathbf{x}^{(2m)}\}$ outputted by a certain oracle
Parameter: Threshold τ_1, τ_2 and (g, t, n, r) of LDPC-PRC
Output: true or false, where true indicates $\mathscr{X}$ are generated by LDPC-PRC and false otherwise

Initialize the counter $S \leftarrow 0$
$\mathscr{Y} \leftarrow \{\mathbf{y}^{(i)} | \mathbf{y}^{(i)} = \mathbf{x}^{(i)} + \mathbf{x}^{(i+m)}, 1 \le i \le m\}$
$\mathscr{H}_t \xleftarrow{\$} \{\mathbf{h} \in \mathbb{F}_2^n | w_H(\mathbf{h}) = t\}$
Construct a subset $\mathscr{H}' \subset \mathscr{H}_t$ with $|\mathscr{H}'| = N_{\text{times}}$
for all $\mathbf{h} \in \mathscr{H}'$ **do**
    $N_{\text{zero}} \leftarrow |\{\mathbf{y}^{(i)} \in \mathscr{Y} | \langle \mathbf{h}, \mathbf{y}^{(i)} \rangle = 0\}|$
    if $N_{\text{zero}} > \tau_1 m$ **then**
        $S \leftarrow S + 1$
    end if
end for
if $S > \tau_2$ **then**
    return true
else
    return false
end if

We then illustrate how Attack-IV works. We will also prove that, with $m = 2\log^2 n$, $N_{\text{times}} = O(\binom{n}{t}/r)$, and appropriate thresholds τ_1, τ_2, the advantage of Algorithm 11 is a constant greater than $1/2$.

One-Time Pad Cancellation. For each watermarked codeword generated by the public key $(\mathbf{G}, \mathbf{z})$, the vector is calculated as $\mathbf{x}^{(i)} = \mathbf{G}\mathbf{s}^{(i)} + \mathbf{e}^{(i)} + \mathbf{z}$. By taking pairwise differences of codewords, we have $\mathbf{y}^{(i)} = \mathbf{x}^{(i)} + \mathbf{x}^{(i+m)} = \mathbf{G}(\mathbf{s}^{(i)} + \mathbf{s}^{(i+m)}) + (\mathbf{e}^{(i)} + \mathbf{e}^{(i+m)})$, which cancels the one-time pad $\mathbf{z}$.

Inner-Product Distribution. If the vector $\mathbf{h}$ is sampled from a row of the secret key $\mathbf{P}$, i.e., $\mathbf{hG} = \mathbf{0}$, then $\langle \mathbf{h}, \mathbf{y}^{(i)} \rangle = \langle \mathbf{h}, \mathbf{G}(\mathbf{s}^{(i)} + \mathbf{s}^{(i+m)}) \rangle + \langle \mathbf{h}, \mathbf{e}^{(i)} + \mathbf{e}^{(i+m)} \rangle = \langle \mathbf{h}, \mathbf{e}^{(i)} + \mathbf{e}^{(i+m)} \rangle$. Let $\tilde{\mathbf{e}}^{(\mathbf{i})} = \mathbf{e}^{(i)} + \mathbf{e}^{(i+m)}$. With $\mathbf{e}^{(i)} \leftarrow \text{Ber}(n, \omega)$,

$\mathbf{e}^{(i+m)} \leftarrow \mathrm{Ber}(n, \omega)$, and the independence of $\mathbf{e}^{(i)}, \mathbf{e}^{(i+m)}$, it is obvious that $\tilde{\mathbf{e}}^{(\mathbf{i})} \leftarrow \mathrm{Ber}(n, \omega')$ where $\omega' = 2\omega(1-\omega)$. Since the weight of $\mathbf{h}$ is t, we can calculate the probability by

$$\Pr\left[\langle \mathbf{h}, \tilde{\mathbf{e}}^{(\mathbf{i})} \rangle = 0\right] = \sum_{j \text{ is even}} \binom{t}{j} \cdot \omega'^{j} \cdot (1-\omega')^{t-j} = \frac{1}{2}[1 + (1-2\omega')^t] > \frac{1}{2}.$$

While for each codeword without a watermark, the corresponding probability is $1/2$. So we can set the threshold τ_1 as

$$\tau_1 = \frac{1}{2}[1 + \frac{(1-2\omega')^t}{2}].$$

Distinguishing Advantage. We observe that the counter S may increase for two different reasons. (1) If the enumerated vector $\mathbf{h}$ happens to be a row of the secret key matrix $\mathbf{P}$, then the distribution of $\langle \mathbf{h}, \mathbf{y}^{(i)} \rangle$ is biased, and the condition $N_{\mathsf{zero}} > \tau_1 m$ holds with a high probability. (2) If $\mathbf{h}$ is not a row of $\mathbf{P}$ or codewords $\{\mathbf{x}^{(i)}\}$ are not watermarked, then the distribution of $\langle \mathbf{h}, \mathbf{y}^{(i)} \rangle$ is uniform, and the inequality $N_{\mathsf{zero}} > \tau_1 m$ can only occur due to statistical fluctuations with a negligible probability, i.e.,

$$p_{\text{bias}} := \Pr[N_{\mathsf{zero}} > \tau_1 m] = \sum_{j > \tau_1 m} \binom{m}{j} \left(\frac{1}{2}\right)^m \leq 2^{-\Omega(m)}.$$

When the codewords are watermarked, both cases may occur during the enumeration of $\mathbf{h}$. In particular, with $N_{\text{times}} = O(\binom{n}{t}/r)$, we expect to enumerate at least one $\mathbf{h}$ from the rows of $\mathbf{P}$. As a result, the contribution to the counter S is dominated by the first case. When the codewords are not watermarked, the first case never occurs. The event $S > \tau_2$ can only result from statistical fluctuations.

Therefore, with $\tau_2 = 0$, the TPR for $S > \tau_2$ is a constant for the watermarked distribution. For the unwatermarked distribution, the FPR $= 1 - (1 - p_{\text{bias}})^{N_{\text{times}}} = O(N_{\text{times}} \cdot 2^{-\Omega(m)}) = O(2^{\log^2 n - \Omega(m)})$ is negligible when $m = 2\log^2 n$. This separation gives a distinguishing advantage

$$\mathrm{Adv} = \frac{1}{2}\mathrm{TPR} + \frac{1}{2}(1 - \mathrm{FPR}) = \frac{1 + \mathrm{TPR}}{2} - \mathsf{negl}(\lambda),$$

which is a constant greater than $1/2$.

A.2 Attack-V: Parity-2 Distinguisher

We next adapt Attack-II to the setting where the public keys are not available. The attack does not rely on explicitly identifying weak rows, but rather on enumerating parity-2 vectors that implicitly exploit such weaknesses. Recall that a public key $\mathbf{G}$ is considered weak if it contains two identical rows $\mathbf{G}_\alpha = \mathbf{G}_\beta$. Although these duplicated rows are unknown to the attacker, their existence

induces a special parity-2 relation that can be detected through enumeration. In particular, when the enumeration happens to include a parity vector $\mathbf{h} = (h_1, \ldots, h_n)$ such that $h_i = 1$ if $i \in \{\alpha, \beta\}$ and $h_i = 0$ otherwise. For any watermarked codeword,

$$\langle \mathbf{h}, \mathbf{Gs} + \mathbf{e} \rangle = \langle \mathbf{G}_\alpha, \mathbf{s} \rangle + \langle \mathbf{G}_\beta, \mathbf{s} \rangle + \langle \mathbf{h}, \mathbf{e} \rangle = \langle \mathbf{h}, \mathbf{e} \rangle.$$

The last equality follows from the assumption that $\mathbf{G}_\alpha = \mathbf{G}_\beta$. Due to the sparsity of both $\mathbf{h}$ and $\mathbf{e}$, the inner product $\langle \mathbf{h}, \mathbf{e} \rangle$ is biased, and the resulting distinguishing advantage can be analyzed in the same way as in Attack-IV.

Consequently, the procedure of Attack-V follows the same structure as Algorithm 11, where the only difference is that we construct $\mathbf{h}$ with weight 2 instead of t. Therefore, for Attack-V, the FPR remains negligible. However, a non-negligible TPR arises only when the public key is weak. Consequently, the overall TPR should be multiplied by P^{-1}, where P denotes the probability that the public key contains duplicated rows, as computed in Attack-II.

B Prompts of LLMs

- Write a long creative story of 10000 words about the sea.
- Write a long poem about the future of AI.
- Write a Python script to solve the Hanoi Tower problem.
- How to teach my child to cook a perfect steak?
- What is the meaning of life, the universe and everything?
- Rewrite the following sentence: Four score and seven years ago our fathers brought forth on this continent a new nation, conceived in liberty and dedicated to the proposition that all men are created equal.
- What is the capital of France?
- Can you tell me a joke?
- Is there any food recommendation around San Francisco?
- What is the best way to learn history?
- How to make a perfect cup of coffee?
- Can you sing a song for me?
- How to replace a tire on a car?
- Why does a fried chicken cross the street?
- Is there any life on Mars?
- My name of John Doe, I am a software engineer, please write a self-introduction for me.

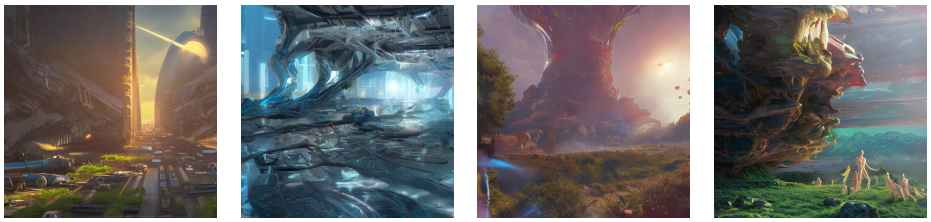
C Generated Text and Image Samples

The sample text with and without watermark are presented in Table 8, and the images with and without watermark are presented in Figure 3 accordingly.

Table 8: LLM's responses to the prompt "Write a 10,000-word creative story about the sea" before and after applying Π_{LLM}, under different temperature settings. Unreadable words are highlighted in red.

	Temperature=1.0	Temperature=1.2	Temperature=1.4	Temperature=1.6	Temperature=1.8
w/o watermark	Okay, so the user is asking me to write a long creative story about the sea, 10,000 words. Hmm, that's a big task. Let me think about how to approach this. **(text remains readable...)**	Alright, so I just came across this request where someone is asking for a 10,000-word creative story about the sea. **(after 194 words)** ensuring that my responses don't再来为ろり **(text with more and more extraneous words and finally unreadable...)**	Okay, so I've been given this task to write a long creative story about the sea, and apparently it needs to be around 10,000 words. That's quite a lot! Hmm, where do I even start? Well, I think writing isls(np Battlefield maybe trying to outline the story first might help. **(mostly unreadable text...)**	Alright, so I have thisCHECK-degree anthropologyNear the Sparrow_contin 绝 urecloggreatomore 是用来表示 otec desperate], çalışmalar horse inended as form to Christoph - **(completely nonsense words...)**	Dadratic Headquarters:fygub-api vein a acute {协调发展 vej 税 Depends sergei unnecessarily _slug örg 匠 userEmail(host Jest 一般都是亲属 VOCSOC Sikh snow DAY.require Success] **(completely nonsense words...)**
w/ watermark	Alright, so the user asked me to write a 10,000-word creative story about the sea. Hmm, that's a big task. Let me think about how to approach this. **(...)** I'll start with a certify特に的故事 that sets the scene. **(text remains readable with few extraneous characters...)**	Okay, so I just got a request where the user asked for a 10,000-word creative story about the sea. **(after 262 words)** Datetime: Narwhal's_script suppose struck enormous filings 嗎? （峰 abet: Is it_blk. **(unreadable text with few coherent sentences...)**	:font-weight: bold;</body> Alright, so the user is asking me to write a 10,000-word creative story about the sea. Hmm, that's a lot, but least I know I can break it down into sections. First, I need to consider the genre. It doesn't specify, but given that example they provided focused on anthropological themes and modern OFFSETverify st 高新区, **(mostly unreadable text...)**	Alright, so I have a user request to write a 10,000-word creative story about the sea. Hmm, that's a prettyComic interchangelobal scale description_be LayoutInflater pe Sch 样板共和国黄瓜 Real 指责 abric story I _DEST on.**(completely nonsense words...)**	Okay, so I've been given an assignment to write a long creative story_PUSH 城区 inability:Get_parentDataSource _fileExists.plugin **(completely nonsense words...)**

Images without watermark



Images with PRC watermarks

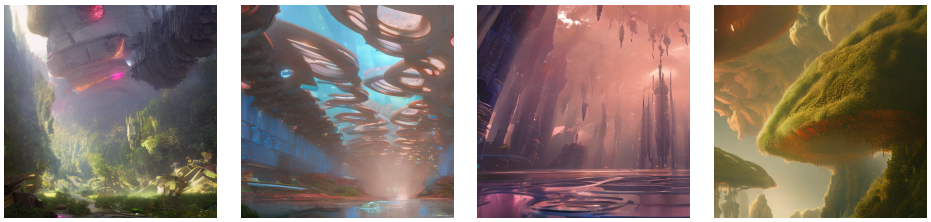


Fig. 3: Images for the prompt "the day that aliens met us, digital art, epic, lighting, color harmony, volumetric lighting, matte painting, chromatic aberration, shallow depth of field, epic composition, trending on artstation". Although the original paper claims that PRC possesses undetectable properties, our proposed attack enables the distinction between images with and without watermarks.